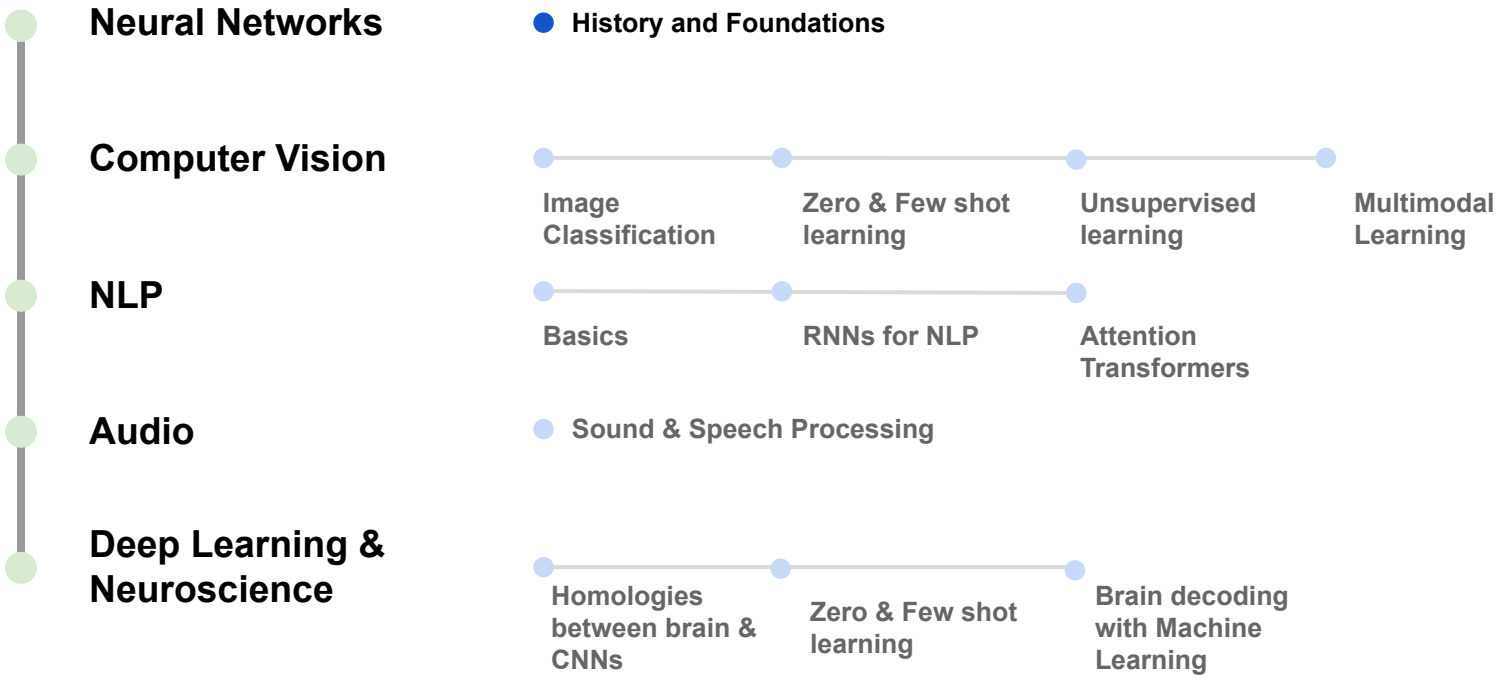
Neural Networks: History and foundation

Léopold Maytié

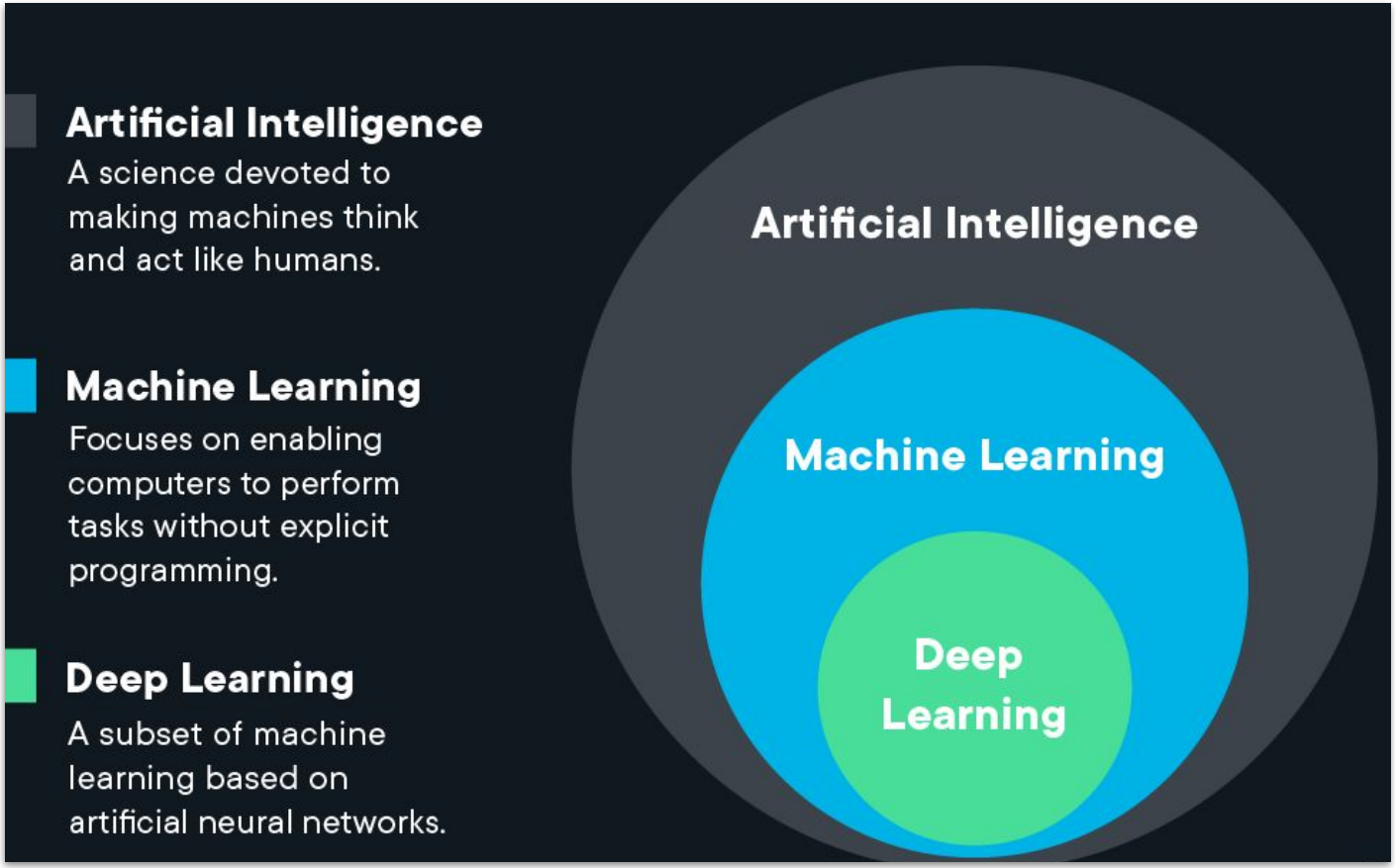
leopold.maytie@univ-tlse3.fr

Artificial and Natural Intelligence Toulouse Institute (ANITI)
March 4th, 2025

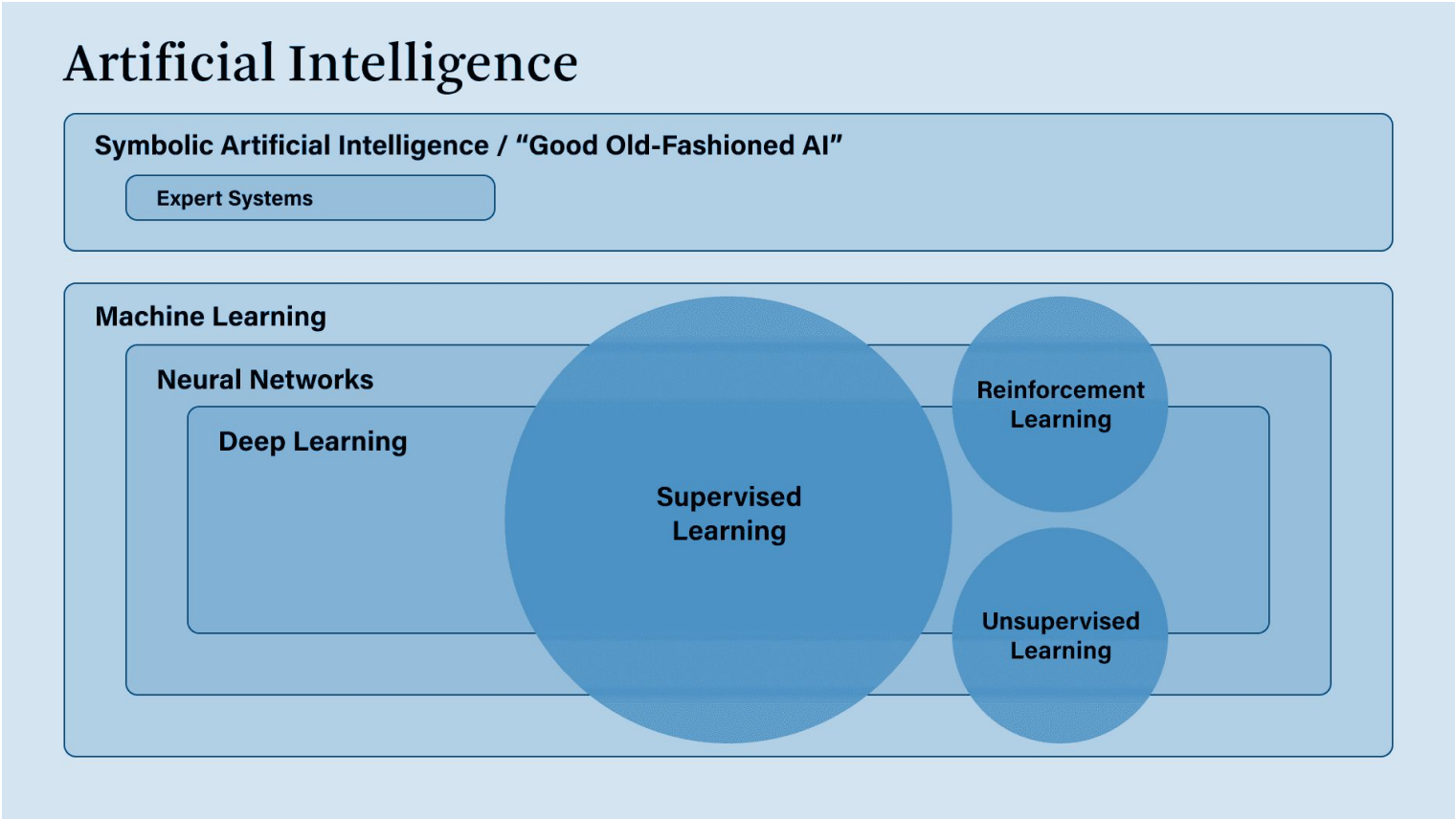
Introduction



Introduction



Introduction



Introduction

Neural networks milestones :

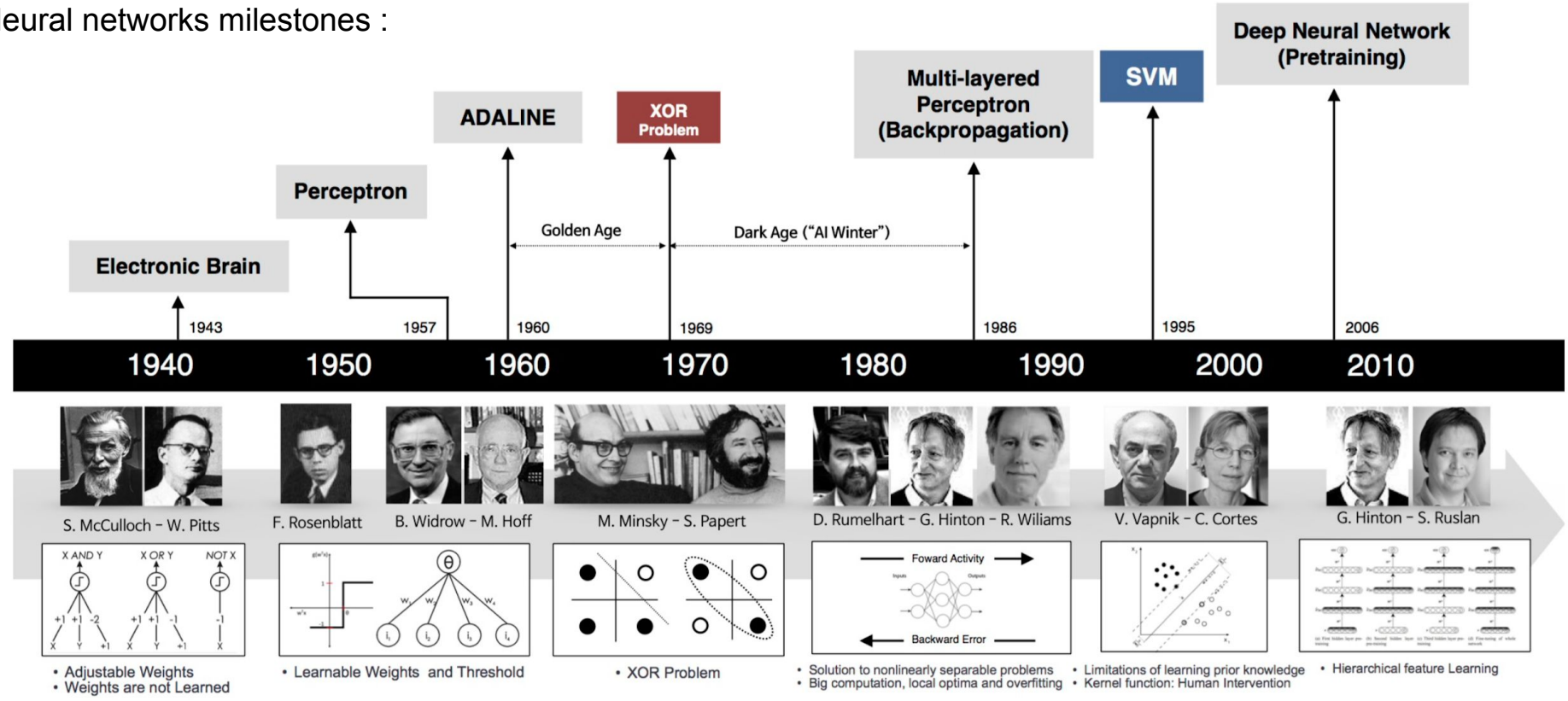
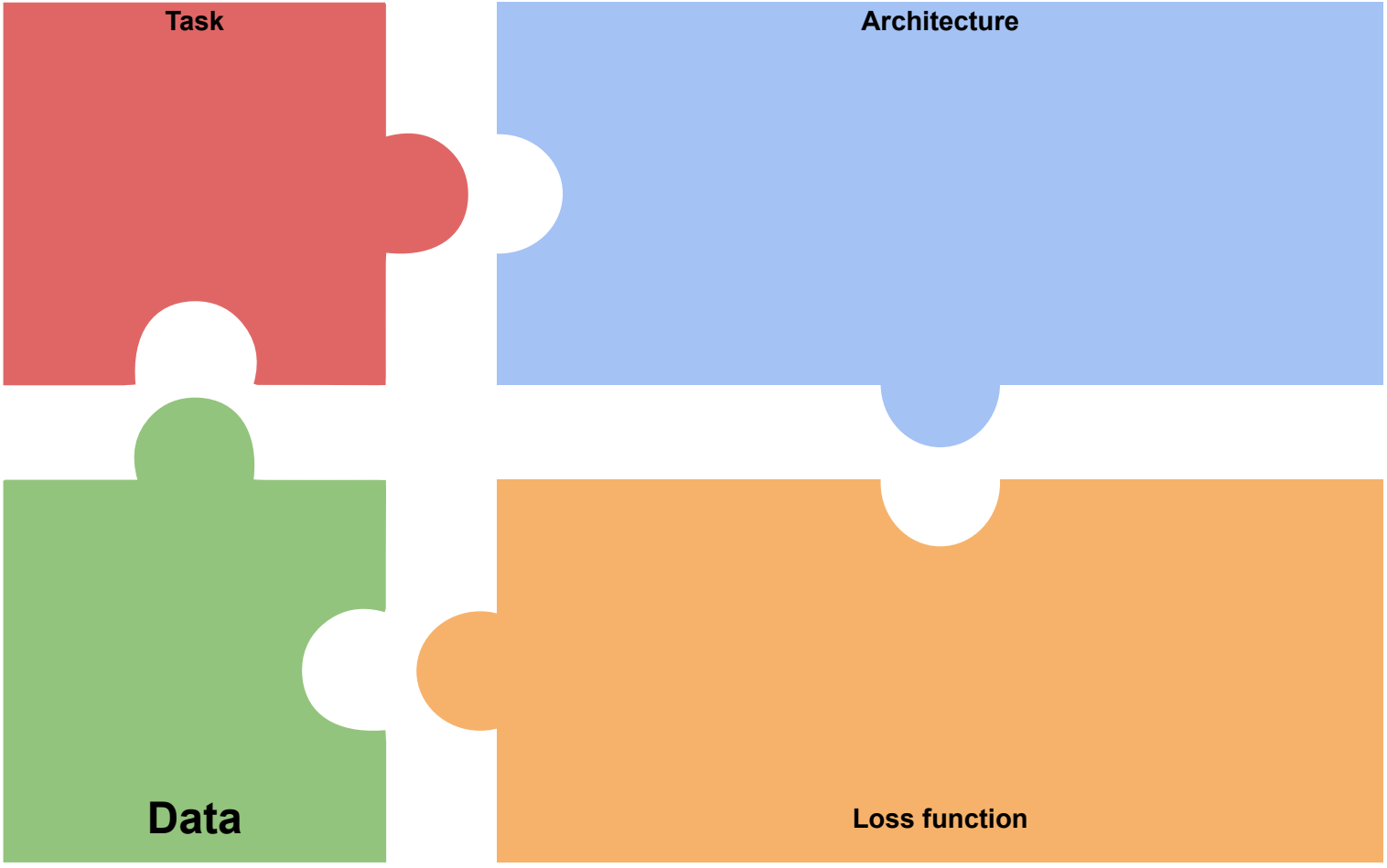
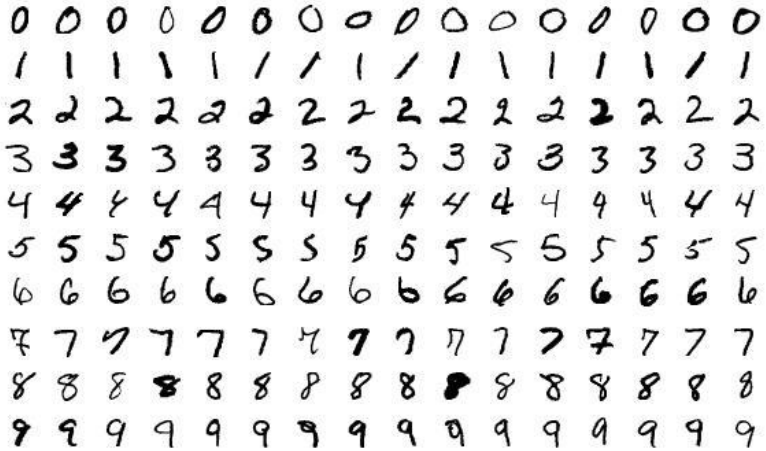


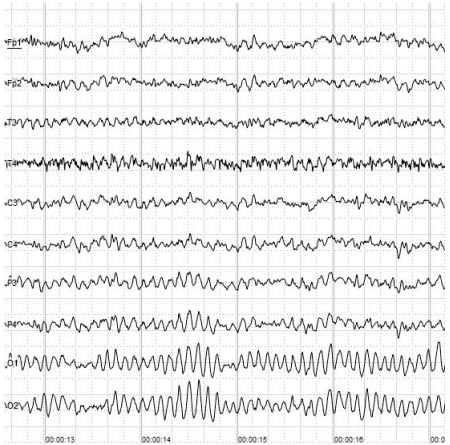
Image source: https://www.slideshare.net/deview/251-implementing-deep-learning-using-cu-dnn

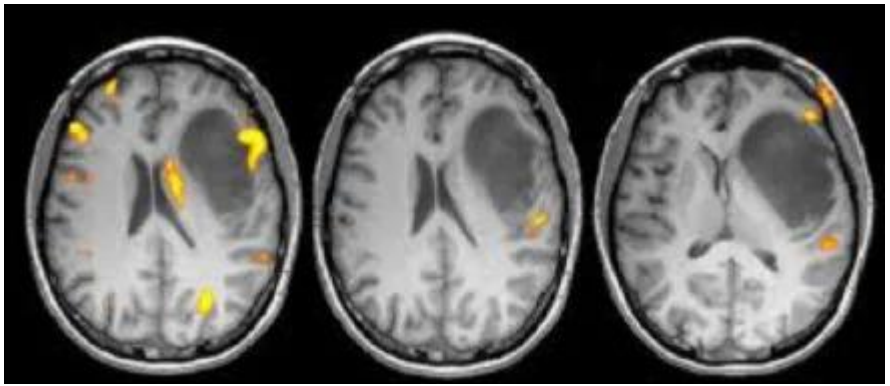
Deep Learning puzzle



Data







Deep Learning puzzle

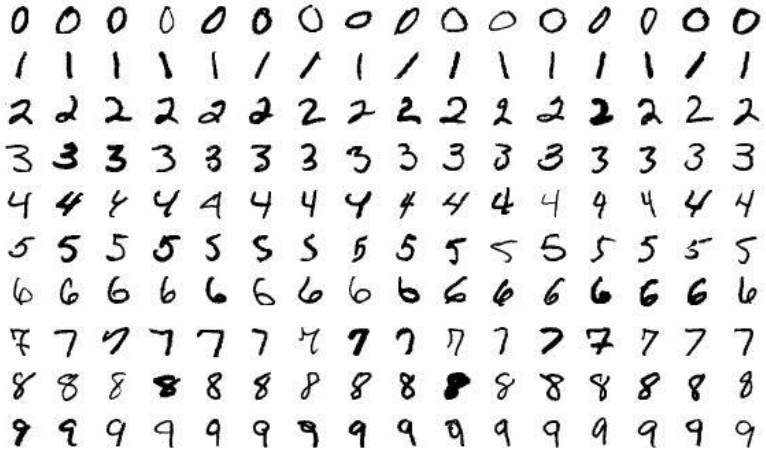
Task

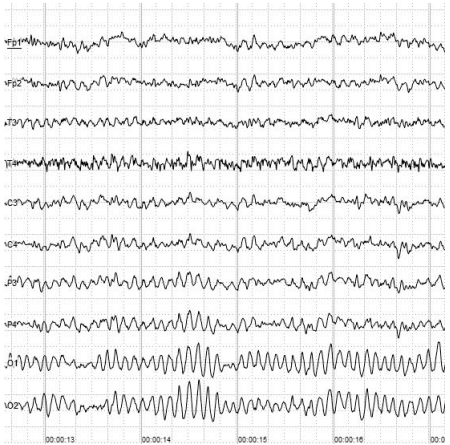
Architecture

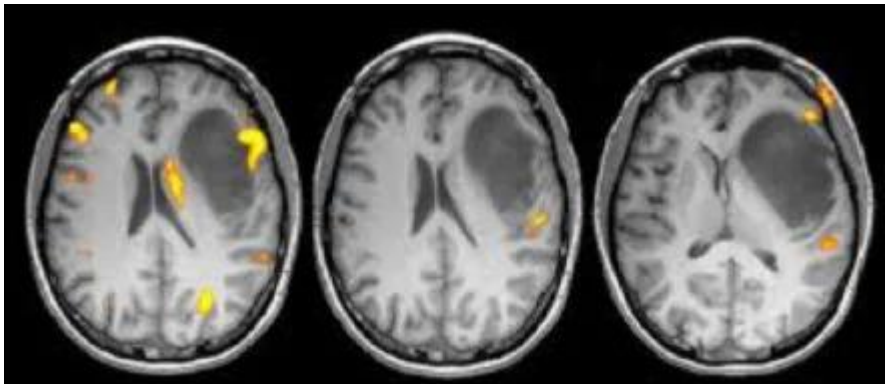
Data

Loss function

Task

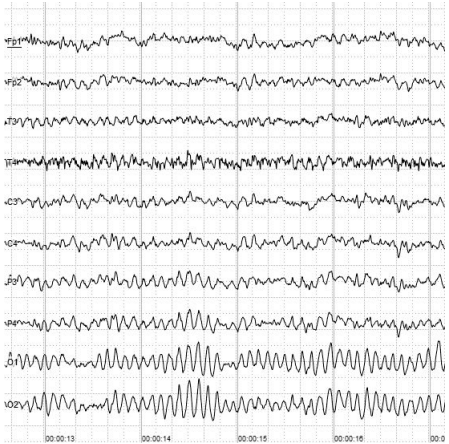


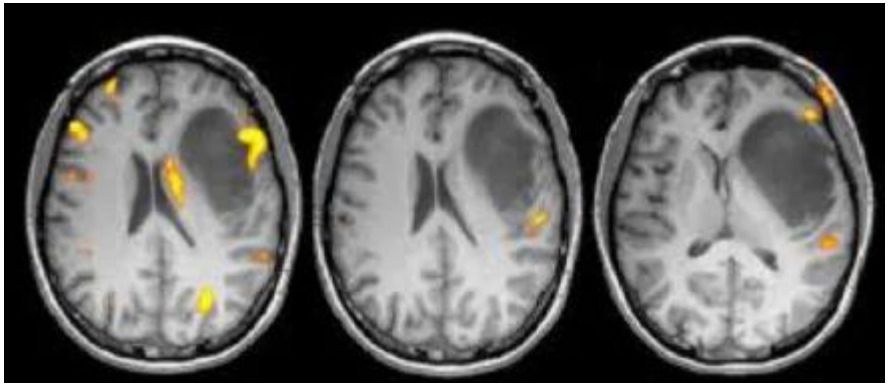




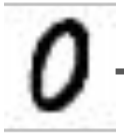
Task

Digit Classification





Task

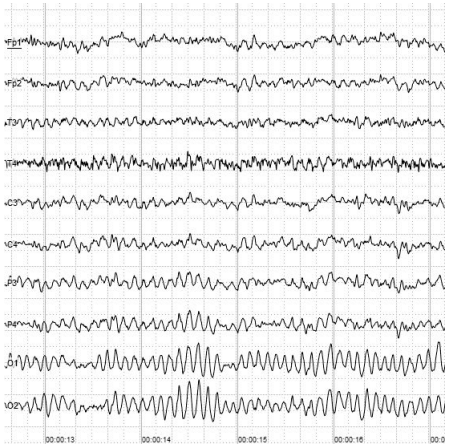


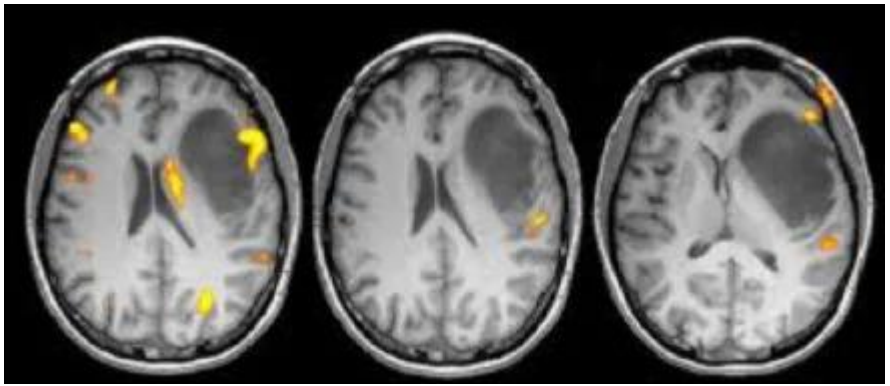
0

Digit Classification

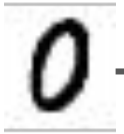
Image Classification

chihuahua or muffin





Task



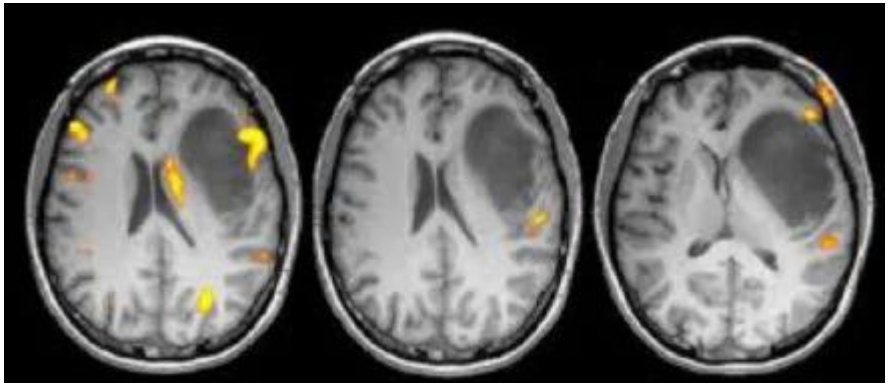
0

Digit Classification

Image Classification

chihuahua or muffin

Predict signal



Task

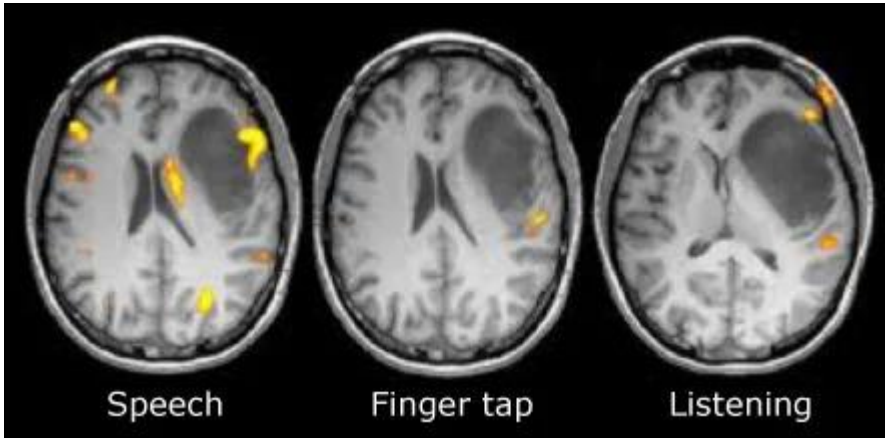
0

Digit Classification

Image Classification

chihuahua or muffin

Predict signal



Find activity

Deep Learning puzzle

Task

0

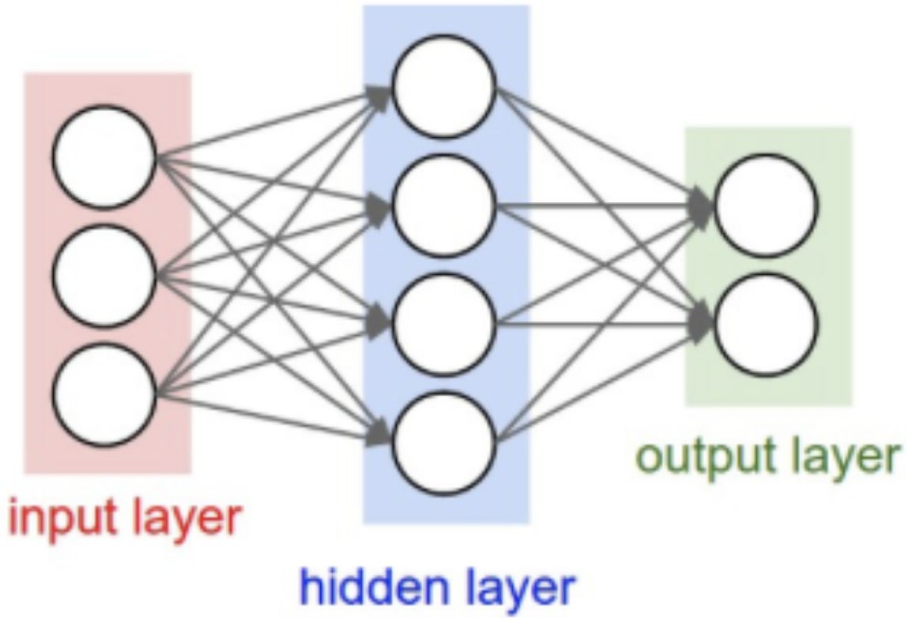
Digit
Classification

Architecture

Data

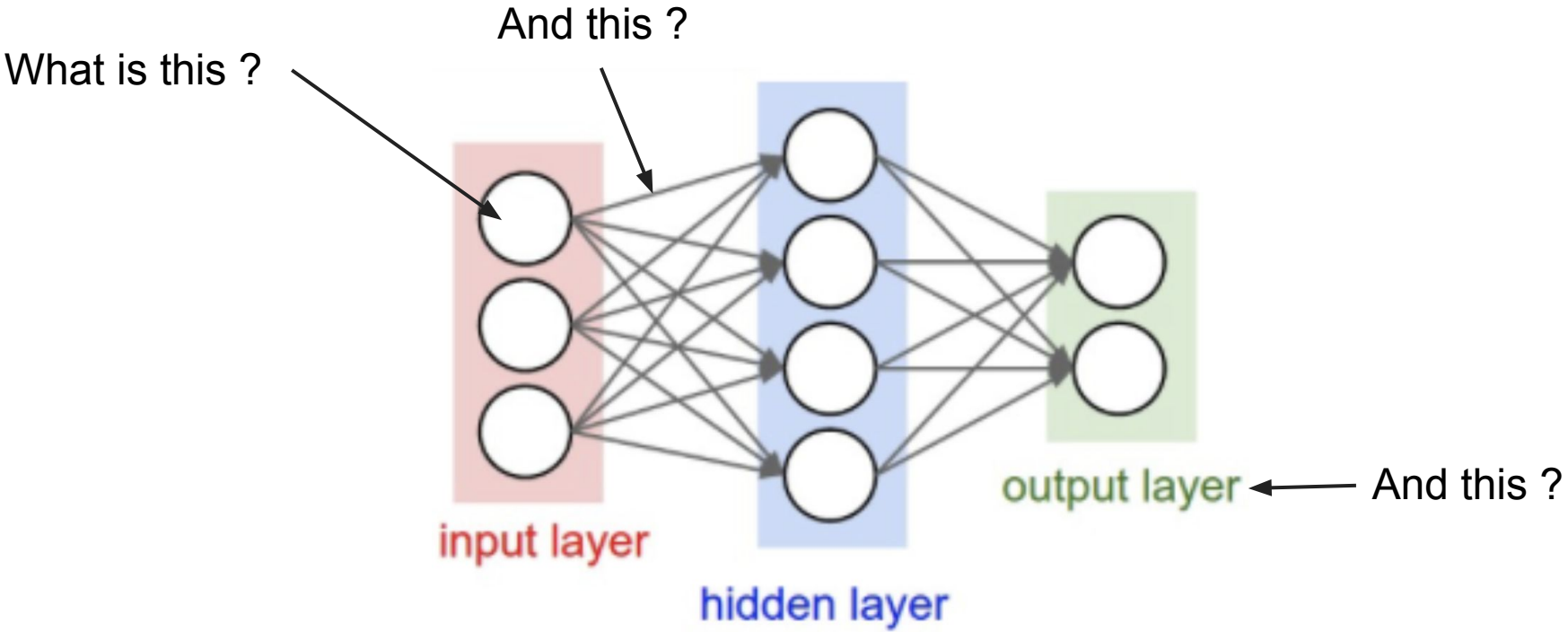
Loss function

Architecture



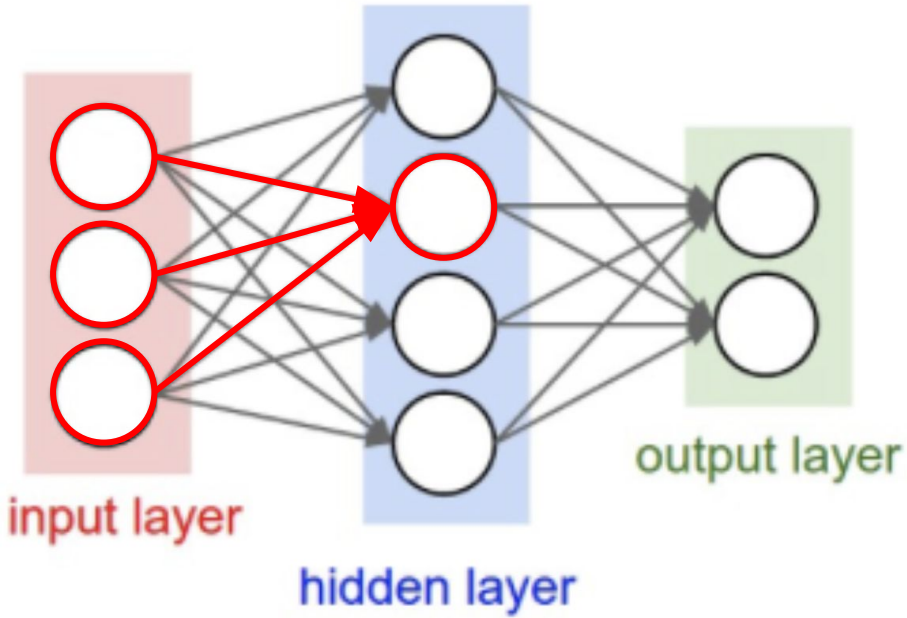
Famous Figure of a Multilayer Perceptron (MLP) network

Architecture



Famous Figure of a Multilayer Perceptron (MLP) network

Architecture

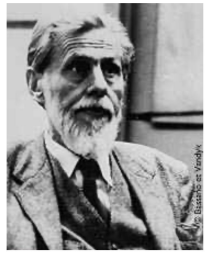


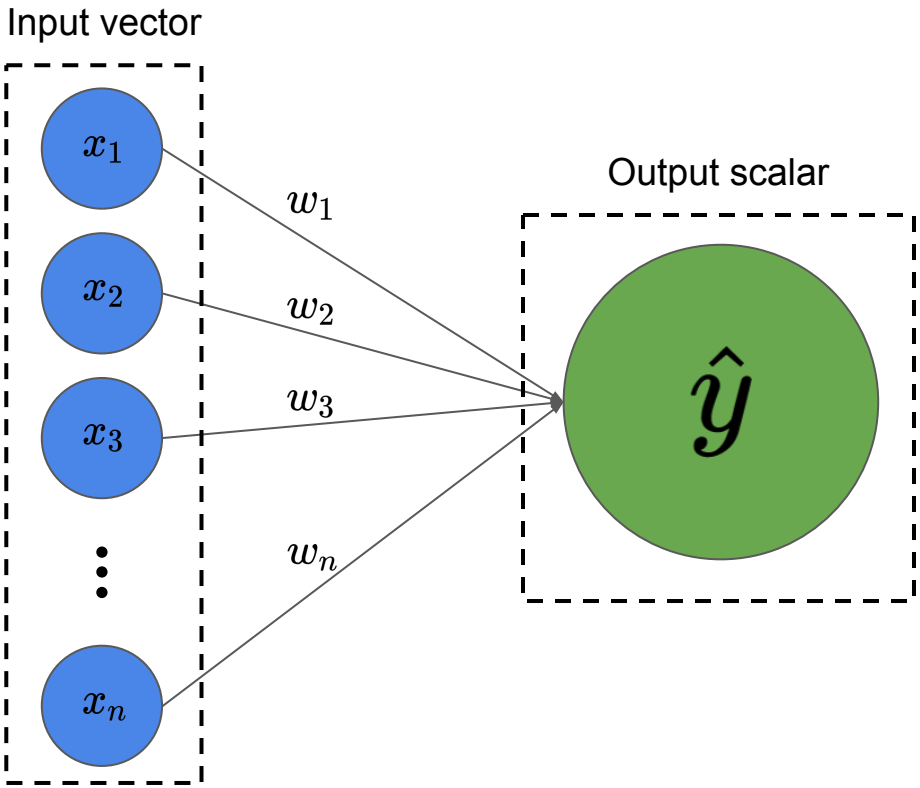
Famous Figure of a Multilayer Perceptron (MLP) network

Let’s do it step by step

Architecture - Formal Neuron, origins

Artificial neuron: McCulloch & Pitt's neuron model (1943)



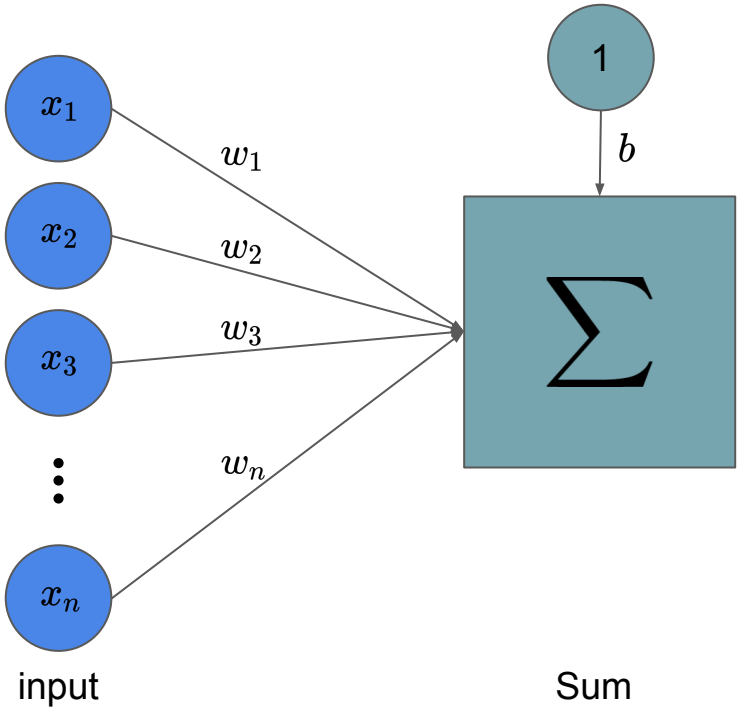


- Input : vector of size n
- Output : scalar
- Parameters w called **weights**

Architecture - Formal Neuron, origins

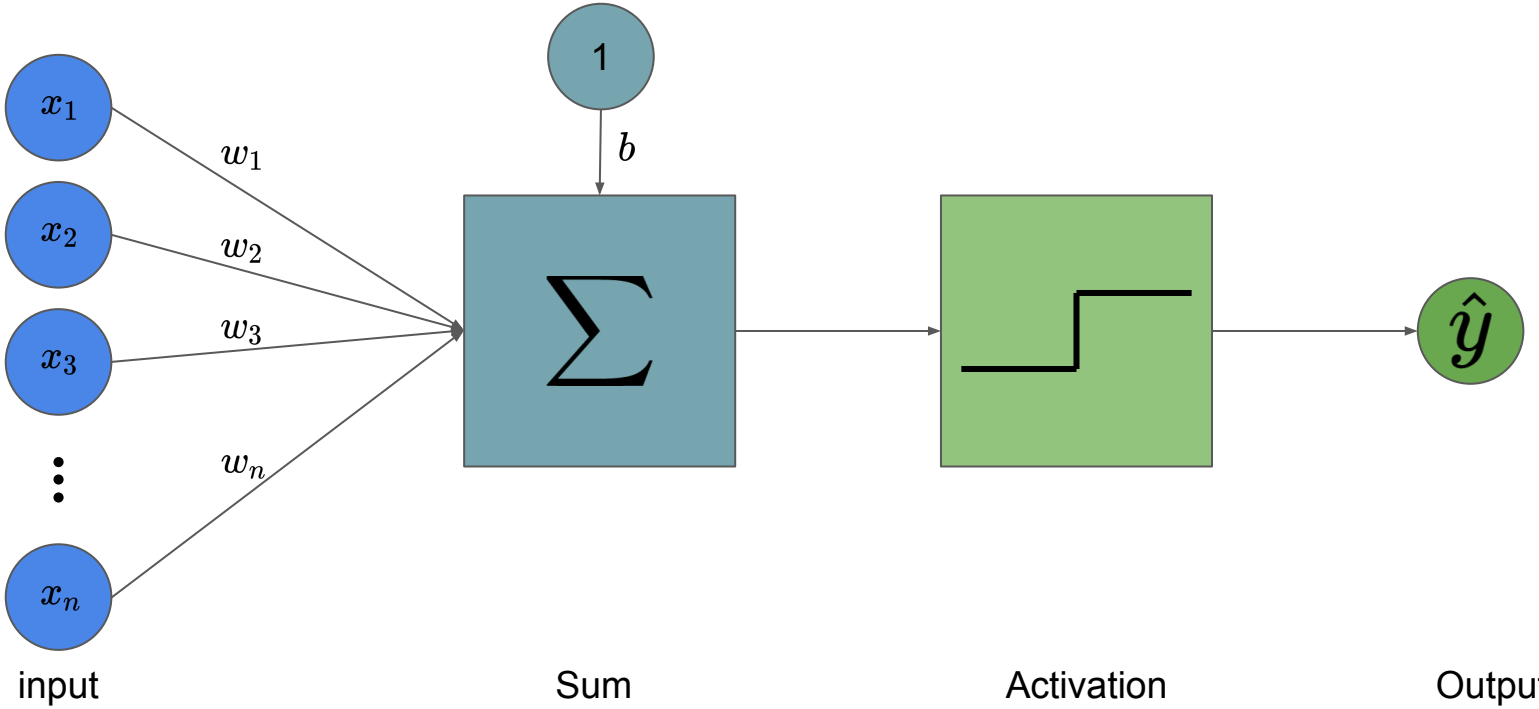
- 1st step : Linear transformation : $s = \boldsymbol{w}^T \boldsymbol{x} + b$

Parameter b called **bias**



Architecture - Formal Neuron, origins

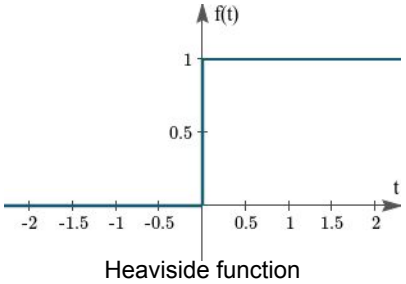
- 1st step : Linear transformation : $s = \boldsymbol{w}^T \boldsymbol{x} + b$
- 2nd step : Non-linear **activation function** : $\hat{y} = f(s)$



Architecture - Formal Neuron, origins

Similarities with biological neurons:

- Choose f as Heaviside function : $H(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{otherwise} \end{cases}$



Heaviside function

- $\hat{y} = 1 \Leftrightarrow \mathbf{w}^{\mathrm{T}}\mathbf{x} \geq -\mathrm{b}$ activated
  $\hat{y} = 0 \Leftrightarrow \mathbf{w}^{\mathrm{T}}\mathbf{x} < -\mathrm{b}$ unactivated

- Biological neurons : produce output if input weighted by synaptic weight exceeds the threshold

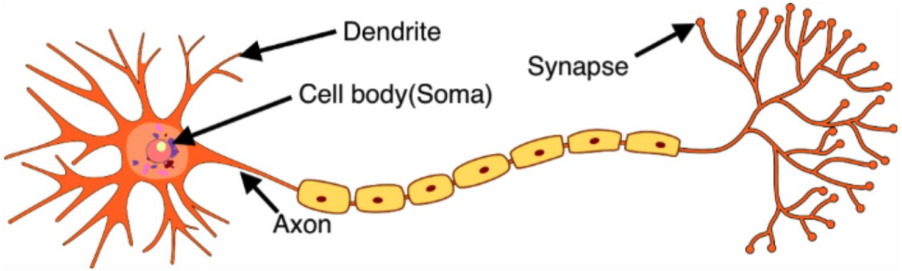
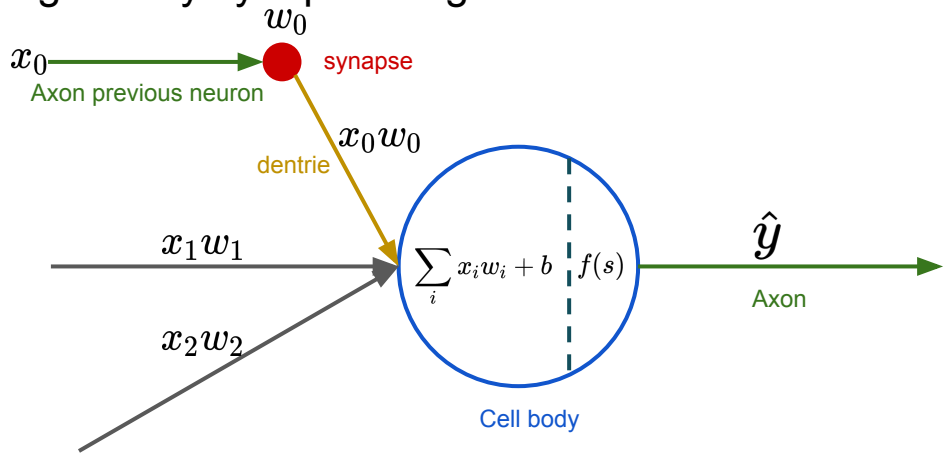
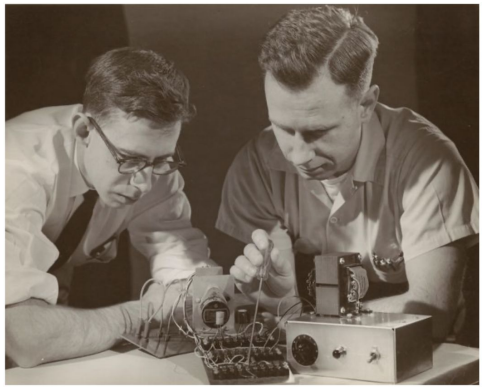


Image source: Wikimedia Commons

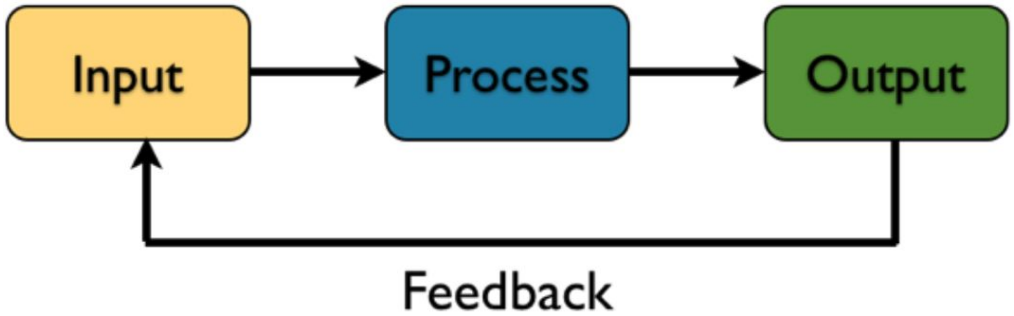


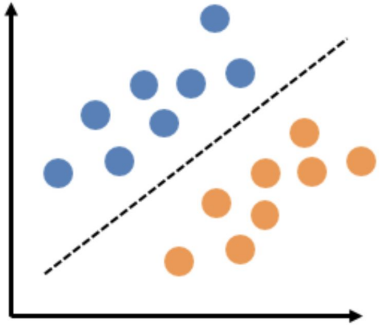
Architecture - Perceptron, first steps in learning algorithm

Rosenblatt, F. (1957). *The perceptron, a perceiving and recognizing automaton Project Para.*



- How a neuron can learn good parameters ?
- The **weights** and **bias** will be learn for a particular task
- Learning is done with the perceptron algorithm





Architecture - Perceptron, first steps in learning algorithm

$$Let\ D = (\langle x_1, y_1 \rangle, \langle x_2, y_2 \rangle, \ldots, \langle x_n, y_n \rangle) \in (\mathbb{R}^m \times \{0, 1\})^n$$

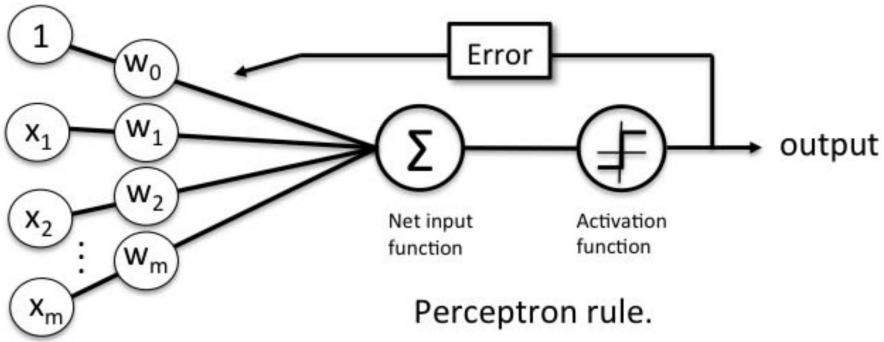
1. Initialise w_i *with random small values*
2. For every training epoch:

   For every sample $\langle x_i, y_i \rangle \in D$:

   (a) $\hat{y} := \sigma(x_i \times w)$ ← Compute output (prediction)

   (b) $err := (y_i - \hat{y}_i)$ ← Compute error

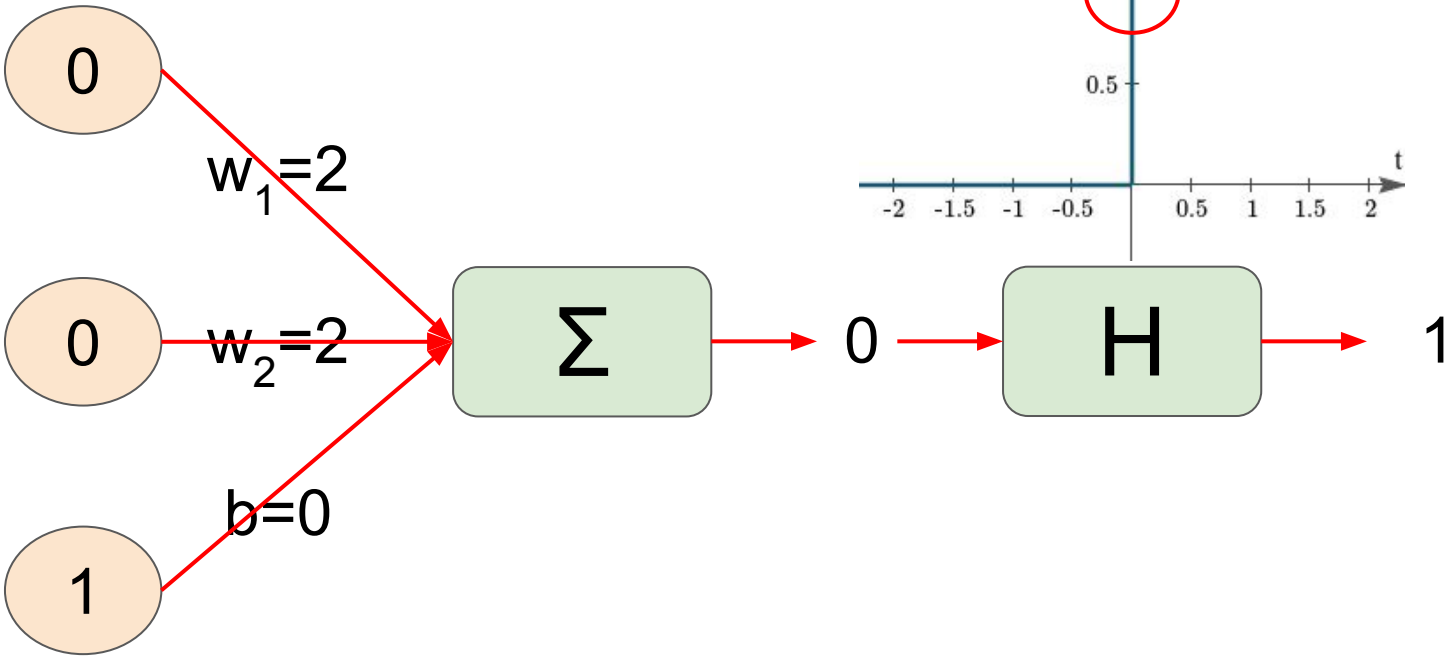
   (c) $w := w + err \times x_i$ ← Update parameters



Perceptron rule.

Architecture - Perceptron, first steps in learning algorithm

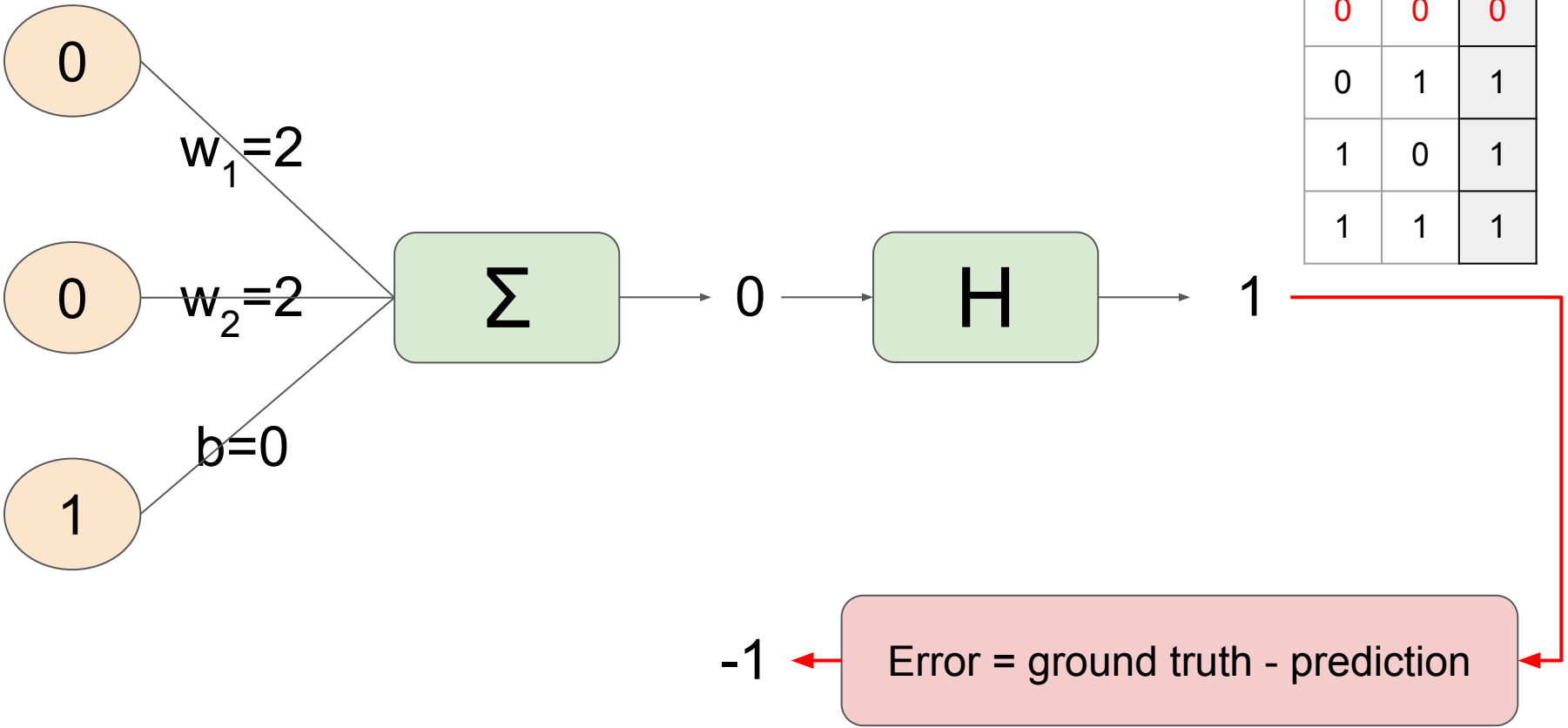
Learning OR gate : predict the value



x_1	x_2	Out
0	0	0
0	1	1
1	0	1
1	1	1

Architecture - Perceptron, first steps in learning algorithm

Learning OR gate : compute error

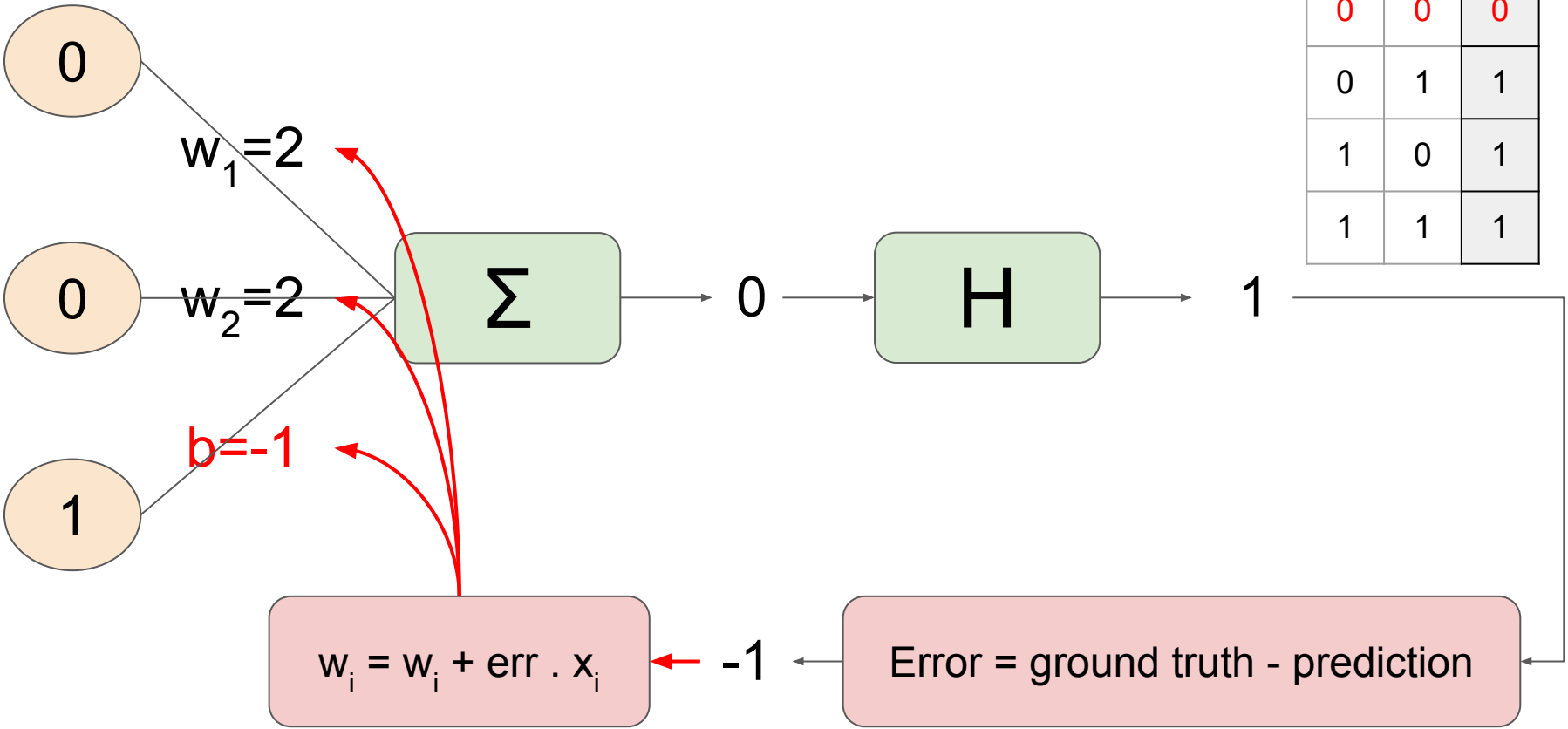


x_1	x_2	Out
0	0	0
0	1	1
1	0	1
1	1	1

Architecture - Perceptron, first steps in learning algorithm

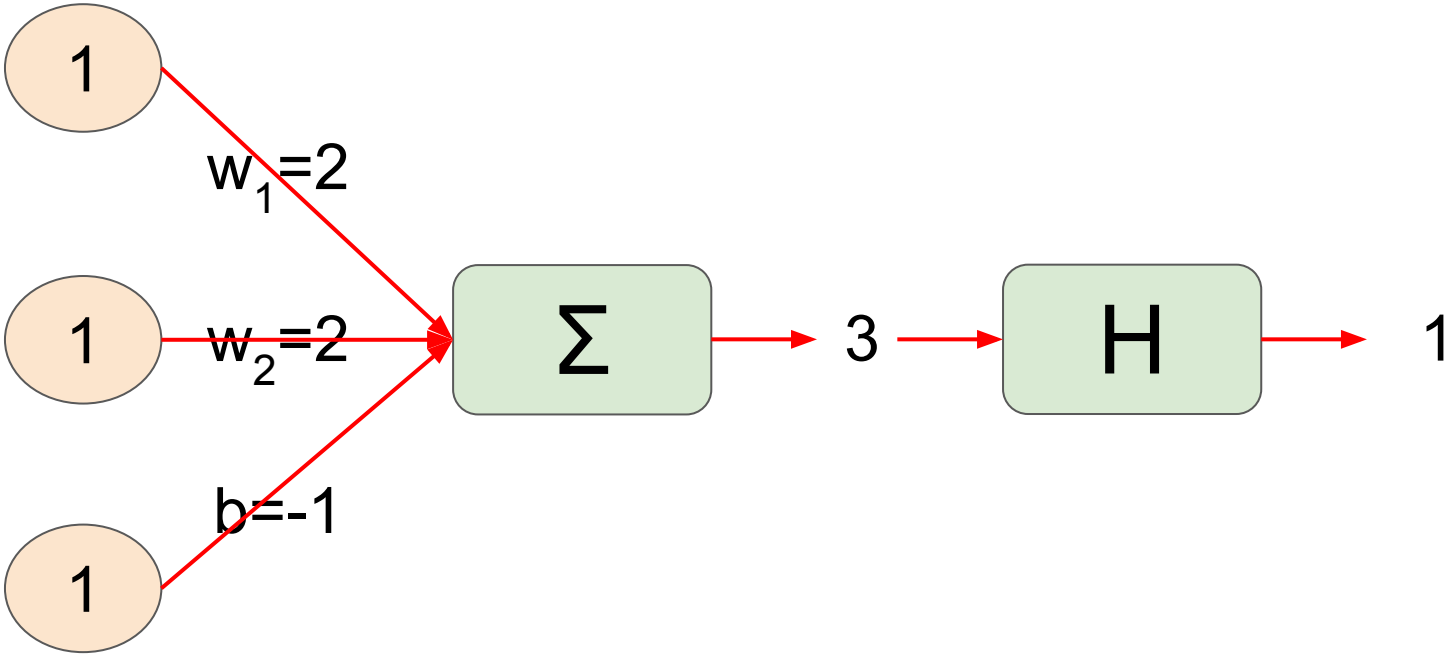
Learning OR gate : update weights

x_1	x_2	Out
0	0	0
0	1	1
1	0	1
1	1	1



Architecture - Perceptron, first steps in learning algorithm

Learning OR gate : second pass

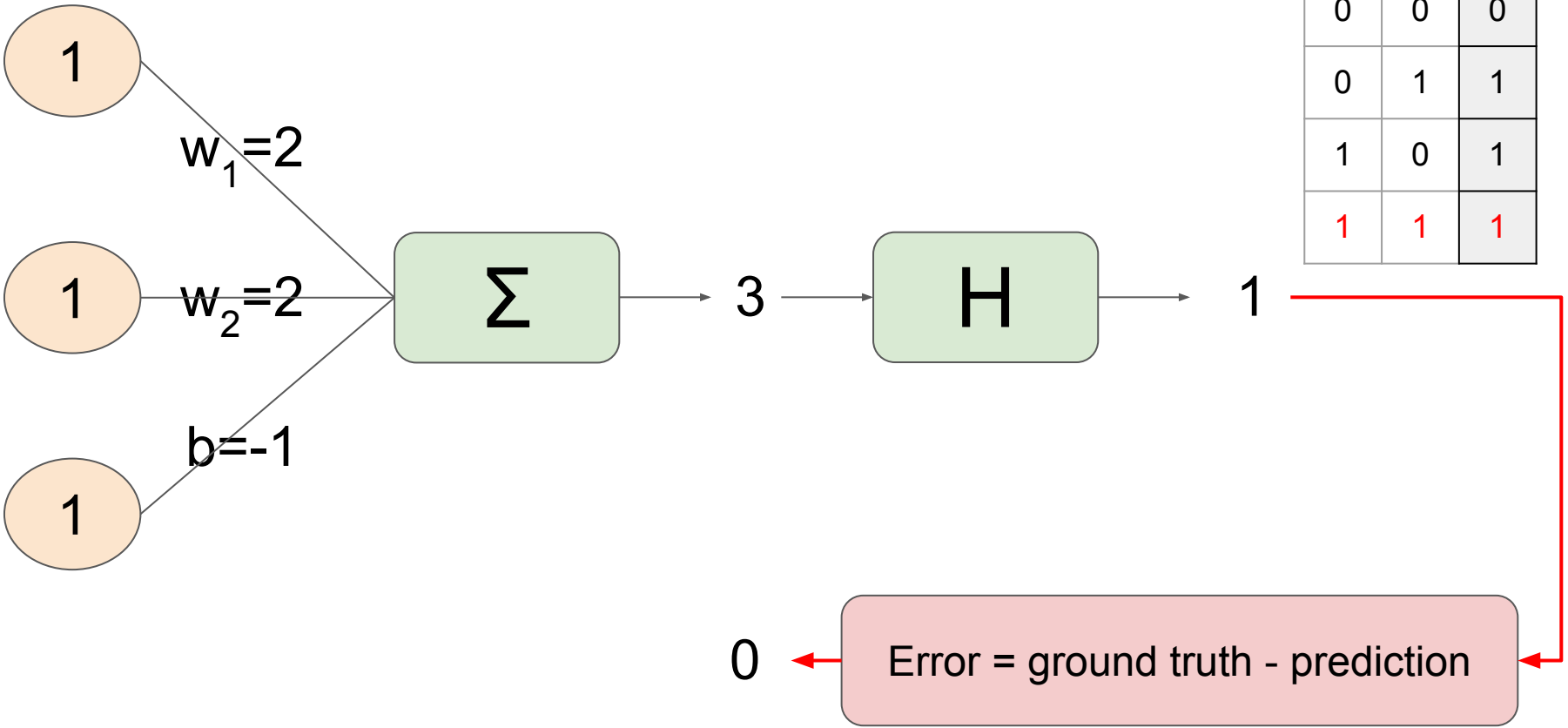


x_1	x_2	Out
0	0	0
0	1	1
1	0	1
1	1	1

Architecture - Perceptron, first steps in learning algorithm

Learning OR gate : compute error

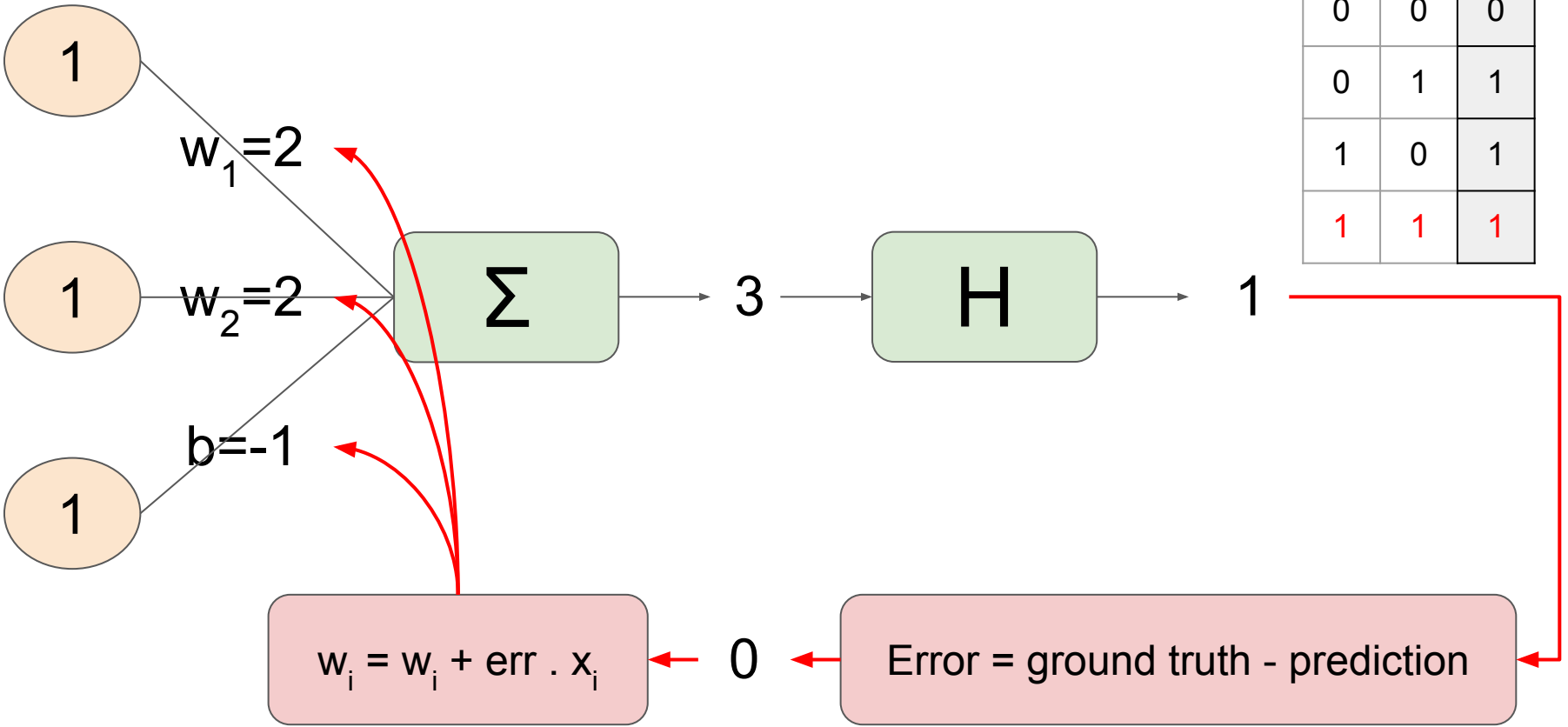
x_1	x_2	Out
0	0	0
0	1	1
1	0	1
1	1	1



Architecture - Perceptron, first steps in learning algorithm

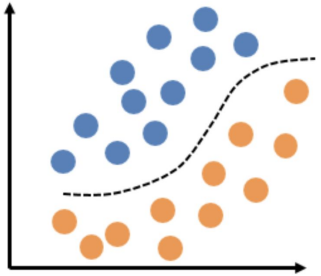
Learning OR gate : compute error

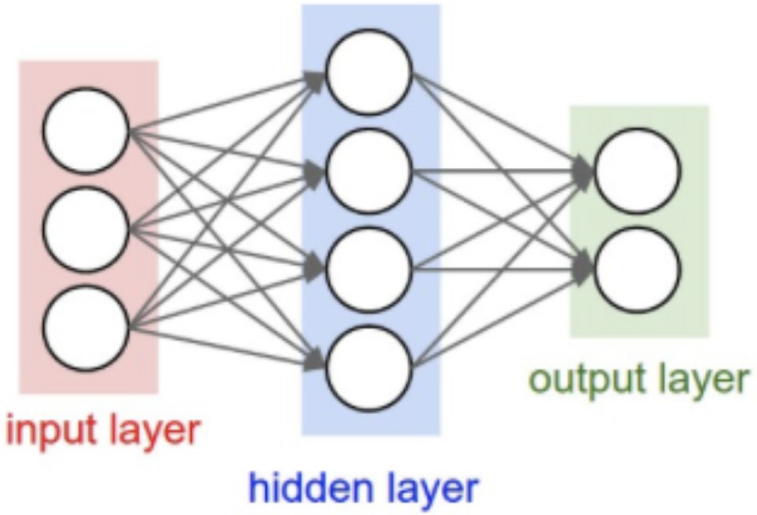
x_1	x_2	Out
0	0	0
0	1	1
1	0	1
1	1	1



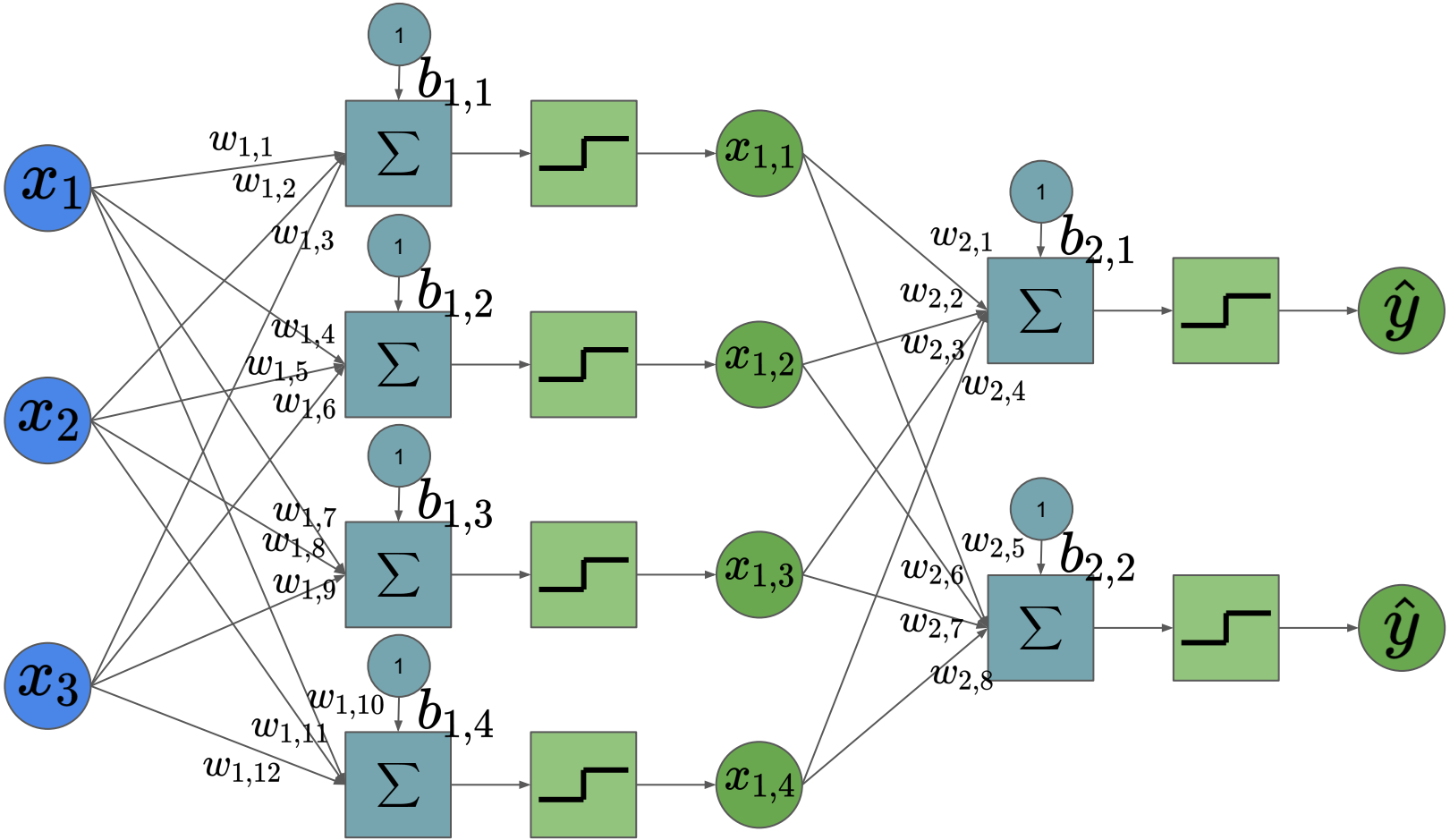
Architecture - Back to MLP

- Composed of: **input** layer, **hidden** layer(s) and an **output** layer.
- Each layer is composed of **neurons**
- Each node (of hidden and output layers) is a neuron that uses a **nonlinear activation function**.
- It can distinguish data that is **not linearly** separable.

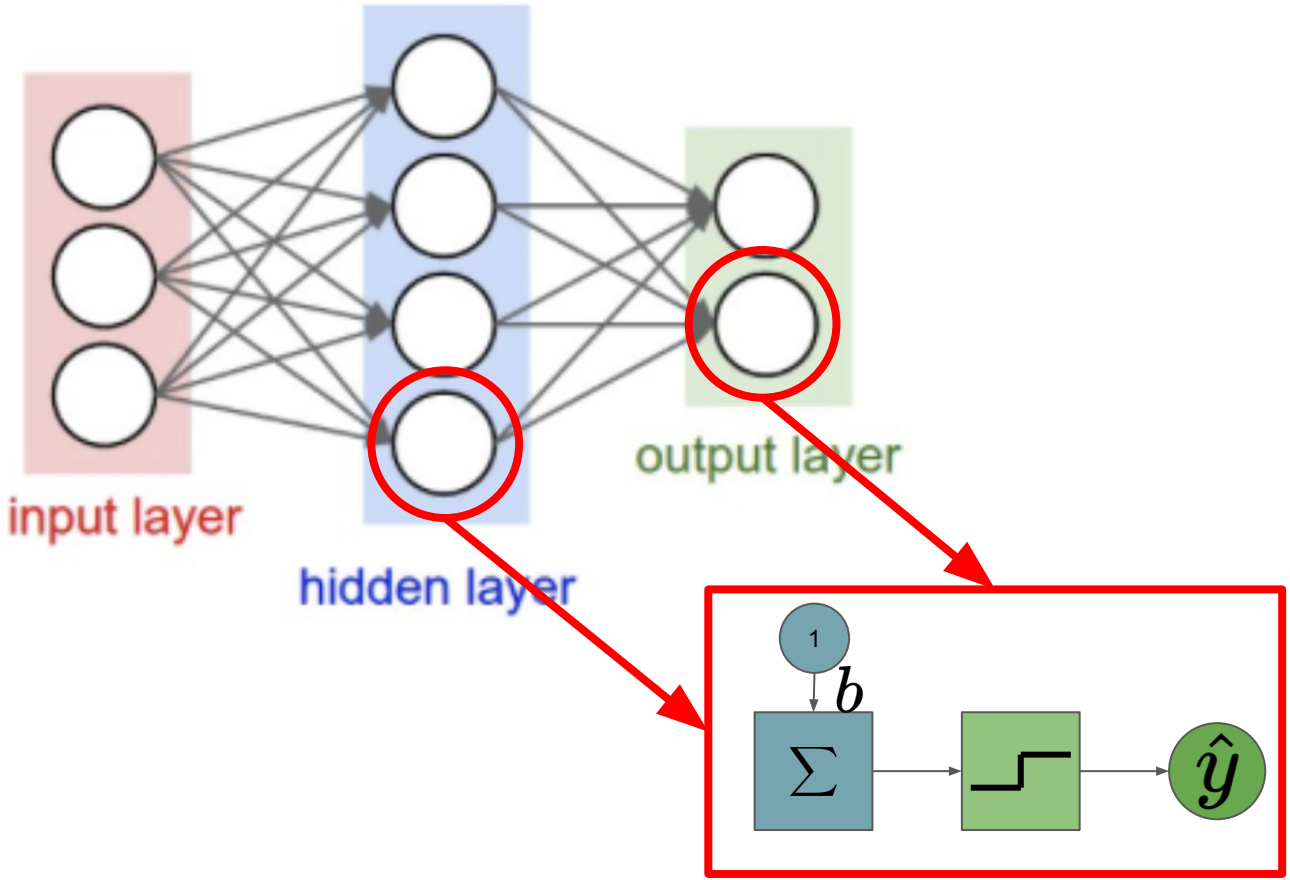




Architecture - Back to MLP



Architecture - Back to MLP

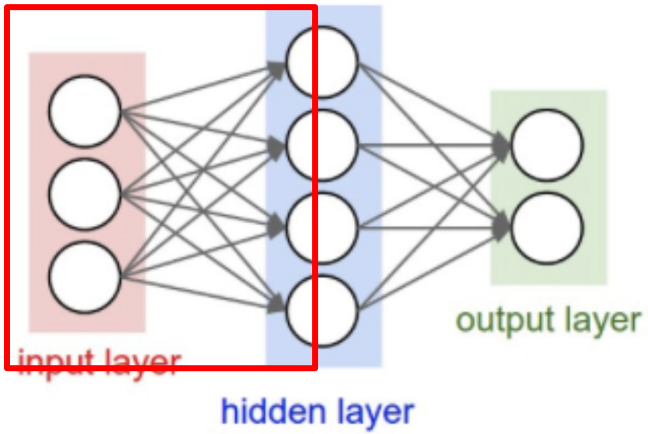


Architecture - Back to MLP

Mathematics behind a Multilayer perceptron :

1- Multiply the input by the weights $\boldsymbol{s} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}$

$$\boldsymbol{s} = \begin{bmatrix} w_{1,1} & w_{1,2} & w_{1,3} \\ w_{1,4} & w_{1,5} & w_{1,6} \\ w_{1,7} & w_{1,8} & w_{1,9} \\ w_{1,10} & w_{1,11} & w_{1,12} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} = \begin{bmatrix} w_{1,1}x_1 + w_{1,2}x_2 + w_{1,3}x_3 + b_1 \\ w_{1,4}x_1 + w_{1,5}x_2 + w_{1,6}x_3 + b_2 \\ w_{1,7}x_1 + w_{1,8}x_2 + w_{1,9}x_3 + b_3 \\ w_{1,10}x_1 + w_{1,11}x_2 + w_{1,12}x_3 + b_4 \end{bmatrix}$$

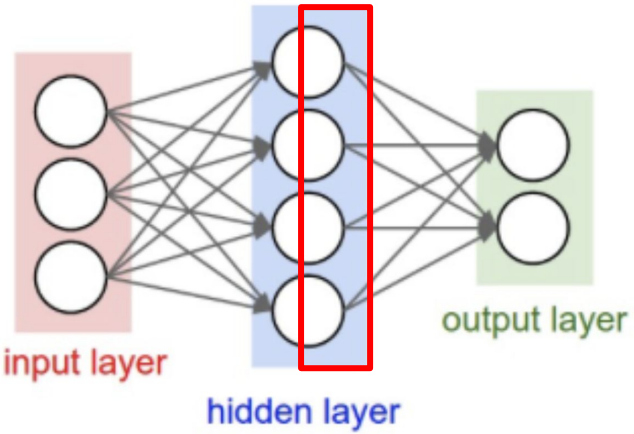


Architecture - Back to MLP

Mathematics behind a Multilayer perceptron :

1- Multiply the input by the weights $$\boldsymbol{s} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}$$

2- Apply non-linearity function $$\begin{bmatrix} x_{11} \\ x_{12} \\ x_{13} \\ x_{14} \end{bmatrix} = f(\boldsymbol{s})$$



Architecture - Back to MLP

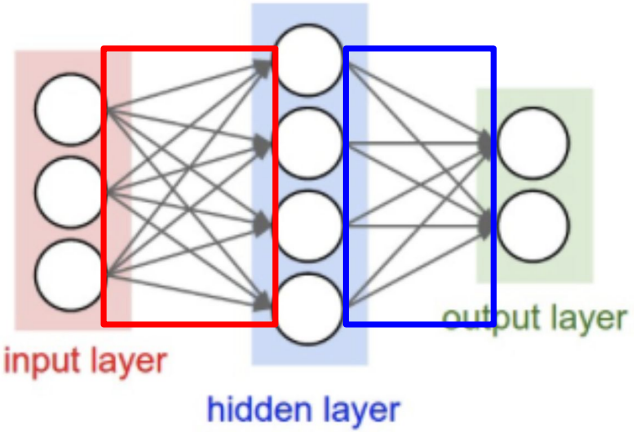
Mathematics behind a Multilayer perceptron :

1- Multiply the input by the weights $$\boldsymbol{s} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}$$

2- Apply non-linearity function $$\begin{bmatrix} x_{11} \\ x_{12} \\ x_{13} \\ x_{14} \end{bmatrix} = f(\boldsymbol{s})$$

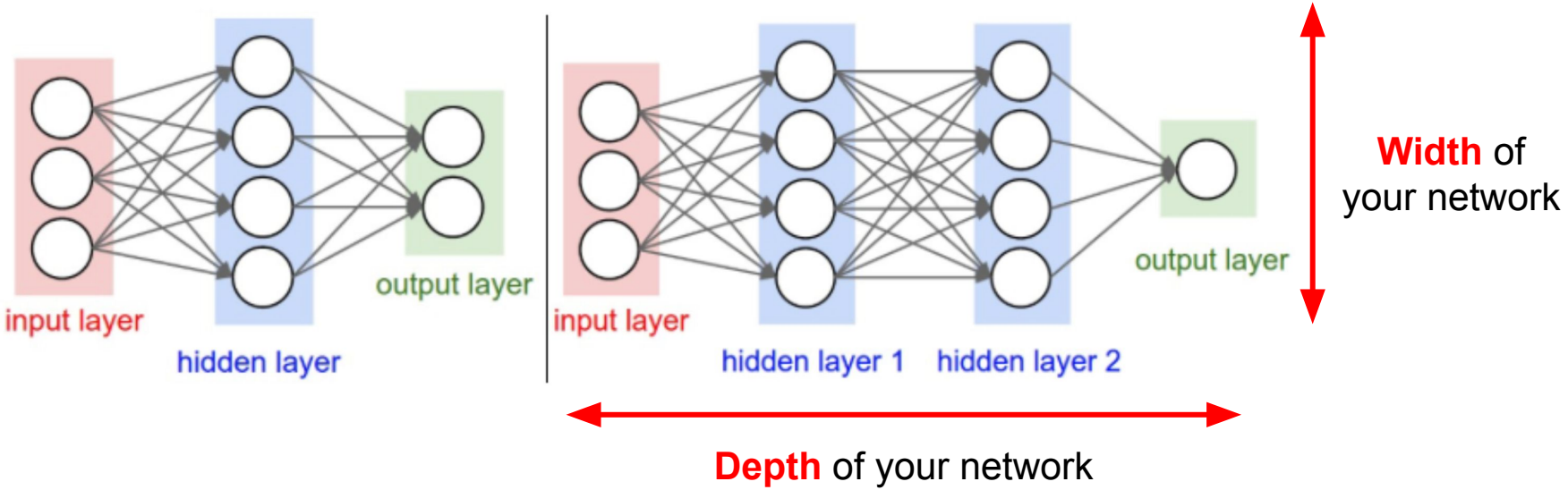
3- Do it for all the layers :

$$\hat{\boldsymbol{y}} = f(\boldsymbol{W_2} f(\boldsymbol{W_1}\boldsymbol{x_1} + \boldsymbol{b_1}) + \boldsymbol{b_2}$$

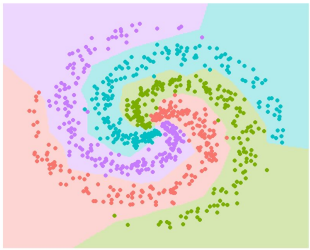


Architecture - Back to MLP

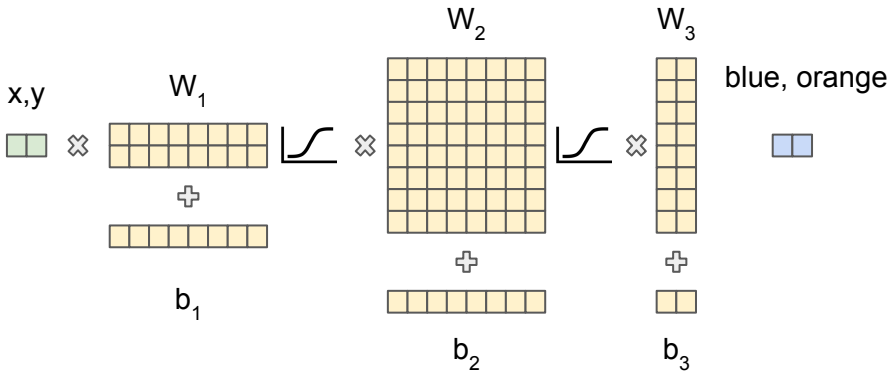
Can add more layers to increase capacity of the network

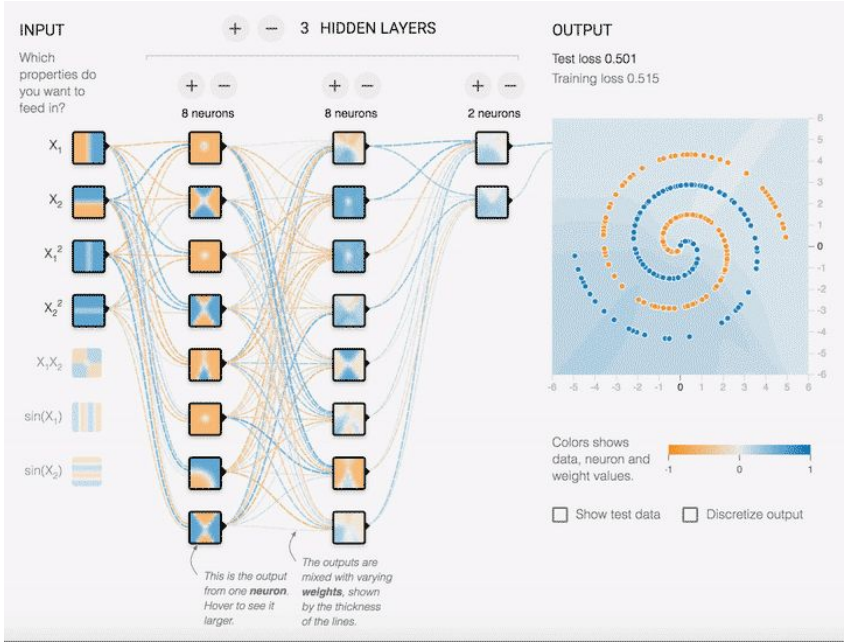


Architecture - How to choose it ?



Multi-layer perceptrons **MLPs** are <u>the standard solution</u> for data with simple structure (ex. tabular data).

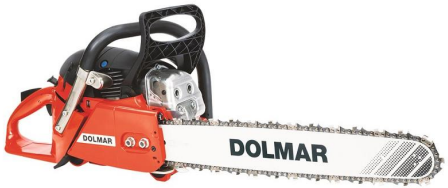


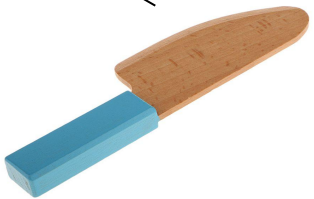


Source: playground.tensorflow.org

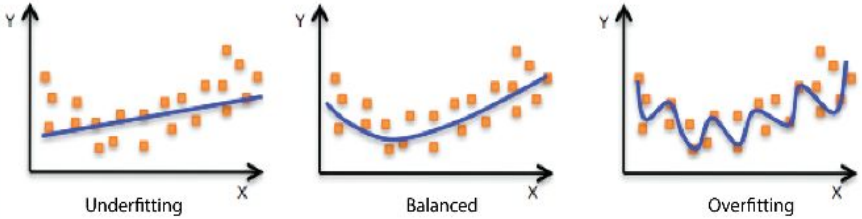
Architecture - How to choose it ?

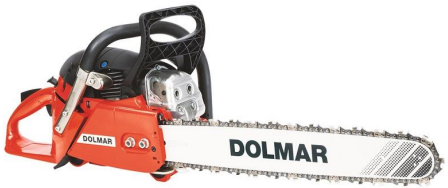
Tool ?





Architecture - How to choose it ?



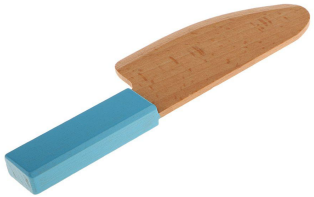


Model too complex :

- Overfitting →Very good at training but bad generalization
- Time and memory inefficient for training

Good model :

- Balanced →Good at training and generalize well
- Efficient time and memory for training



Model too simple :

- Underfitting →Bad on training and generalization
- Very efficient time and memory for training

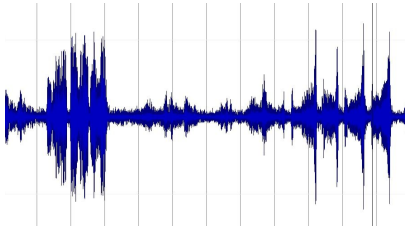
Architecture - Other architectures

- Do MLPs work for all types of data ?
  - Yes, but not efficiently
- **The architecture will depend on the type of data you have**

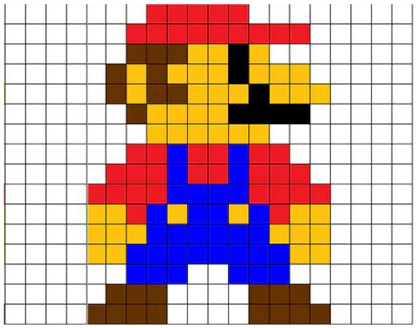
text

I am a student

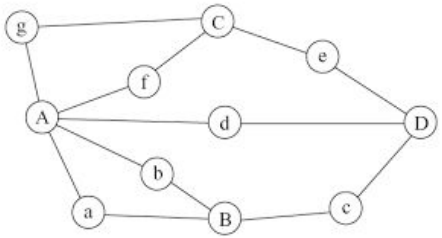
time sequences



images



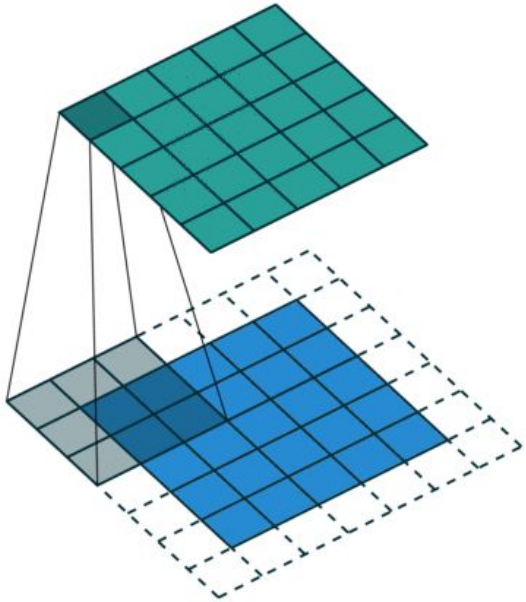
graphs

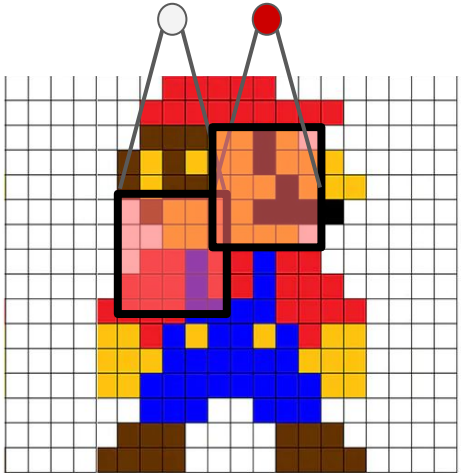


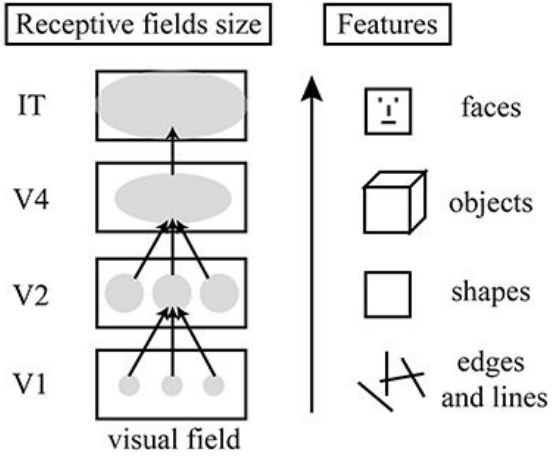
Inductive bias : an architectural assumption or constraint

Architecture - Other architectures

Convolutional Neural Network (CNN)



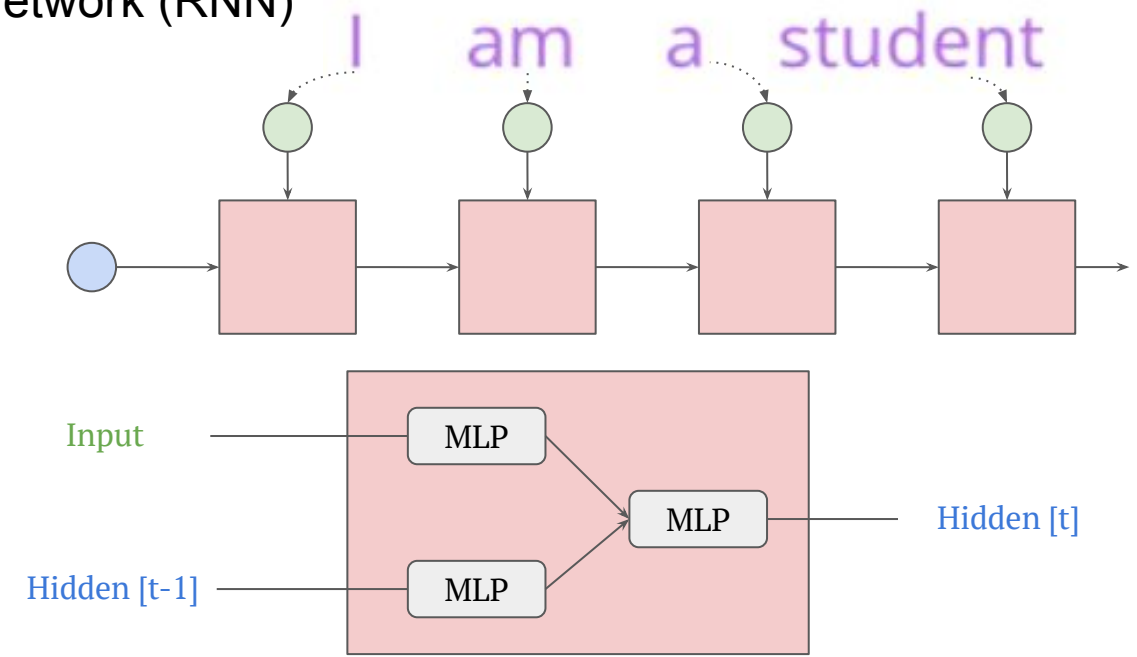




- Catch spatial information
- Better if you have images

Architecture - Other architectures

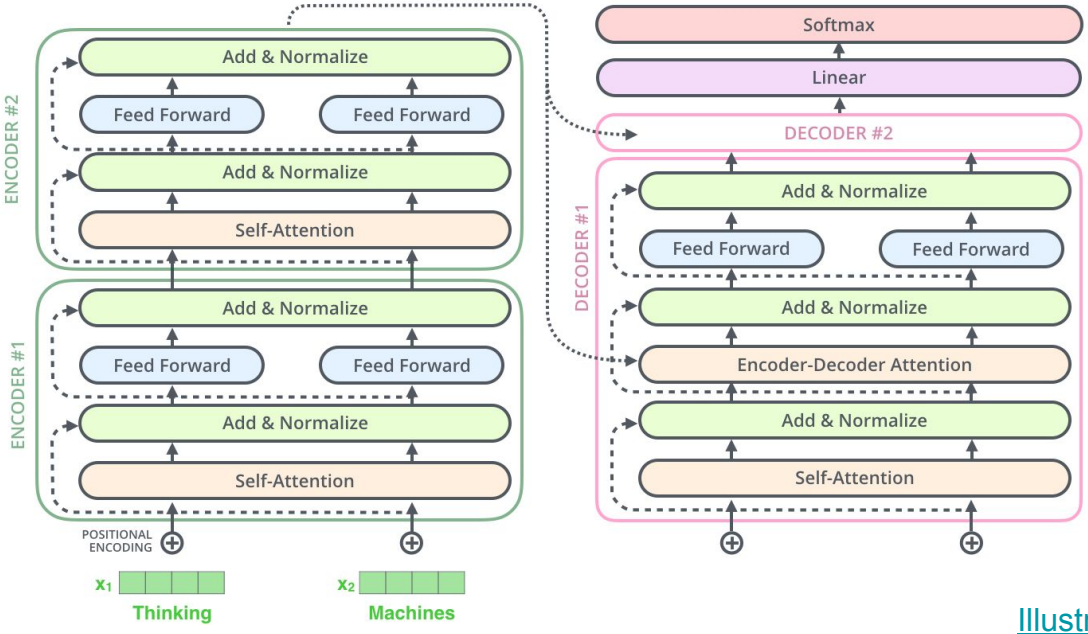
Recurrent Neural Network (RNN)



- Catch temporal information
- Better for sequence (text, EEG, …)

Architecture - Other architectures

Transformer



[Illustrated transformer]

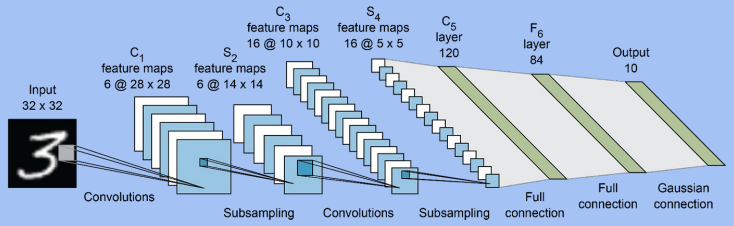
- Very general architecture able to deal with images, text, … at the same time

Deep Learning puzzle

Task

Digit Classification

Architecture

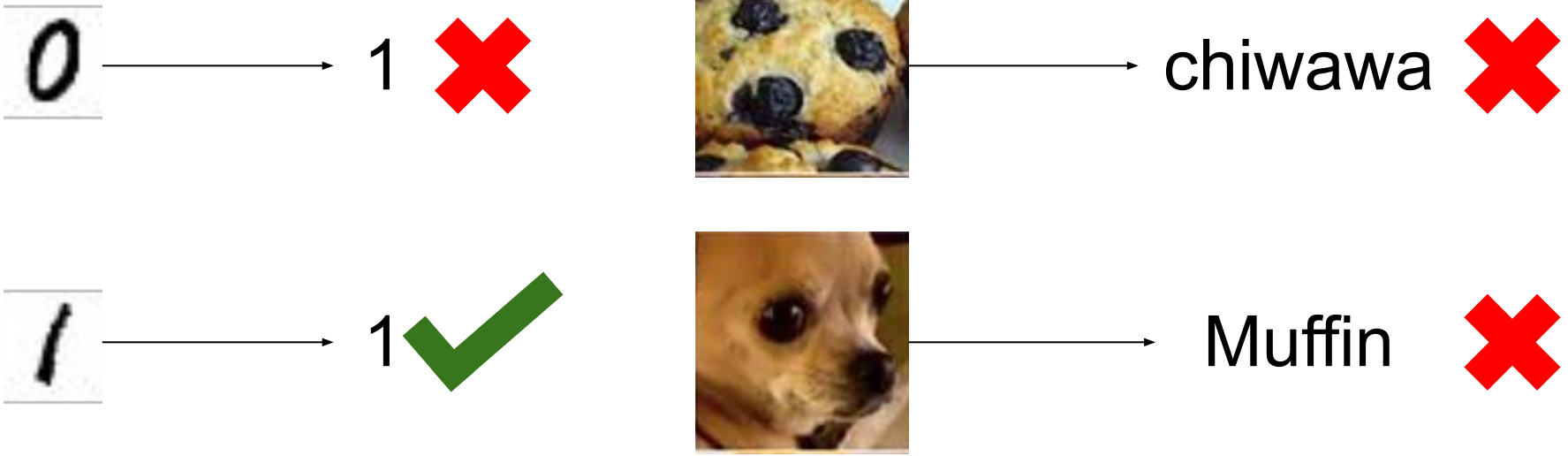


Data

Loss function

Cost Function

What is the goal of your task and how to find good parameters to achieve this goal?



How to tell to your model how much he is bad or good?

Cost Function

What is responsible of the answer of the model for a given input ?

$$\hat{y} = f(\boldsymbol{W_2} f(\boldsymbol{W_1} x_1 + \boldsymbol{b_1}) + \boldsymbol{b_2})$$

Weights and **Biases**

Need to adapt them

Tree ✗✗

W_i, b_i

Chiwawa ✗

W_i, b_i

Define a function telling you how good/bad your model is depending on the weights: **Cost Function**

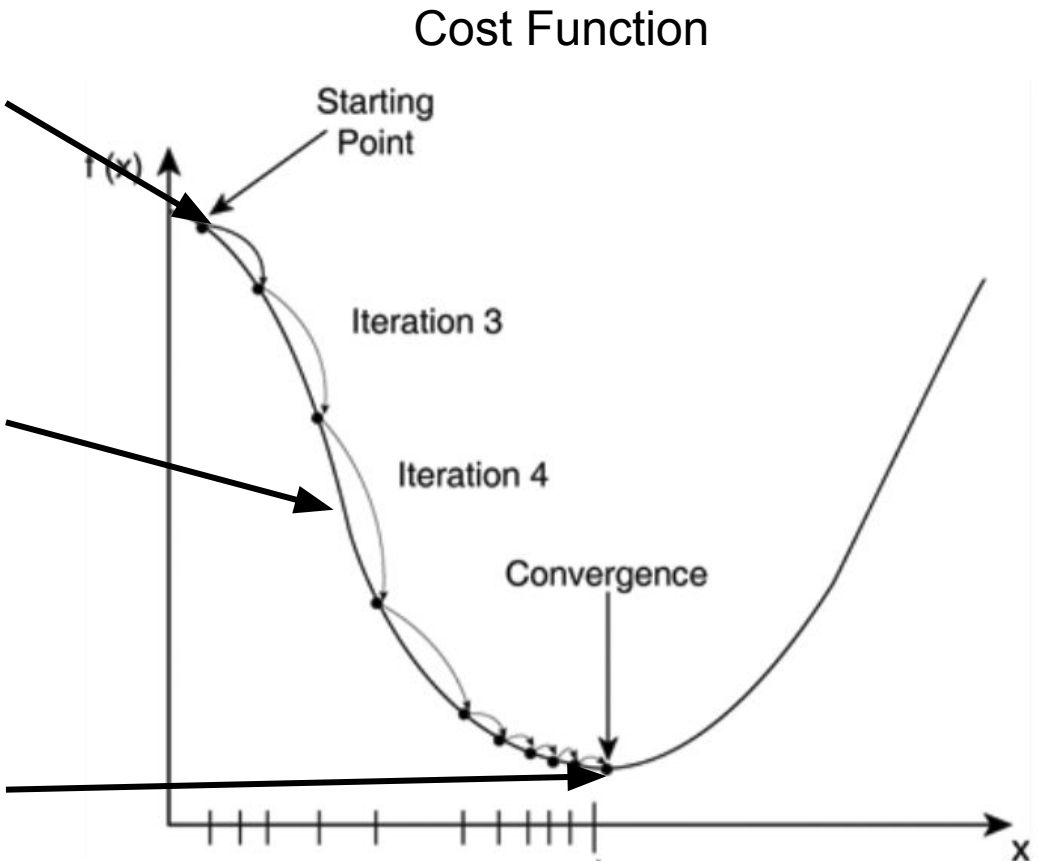
Cost Function = Loss

Cost Function

Chiwawa, Muffin

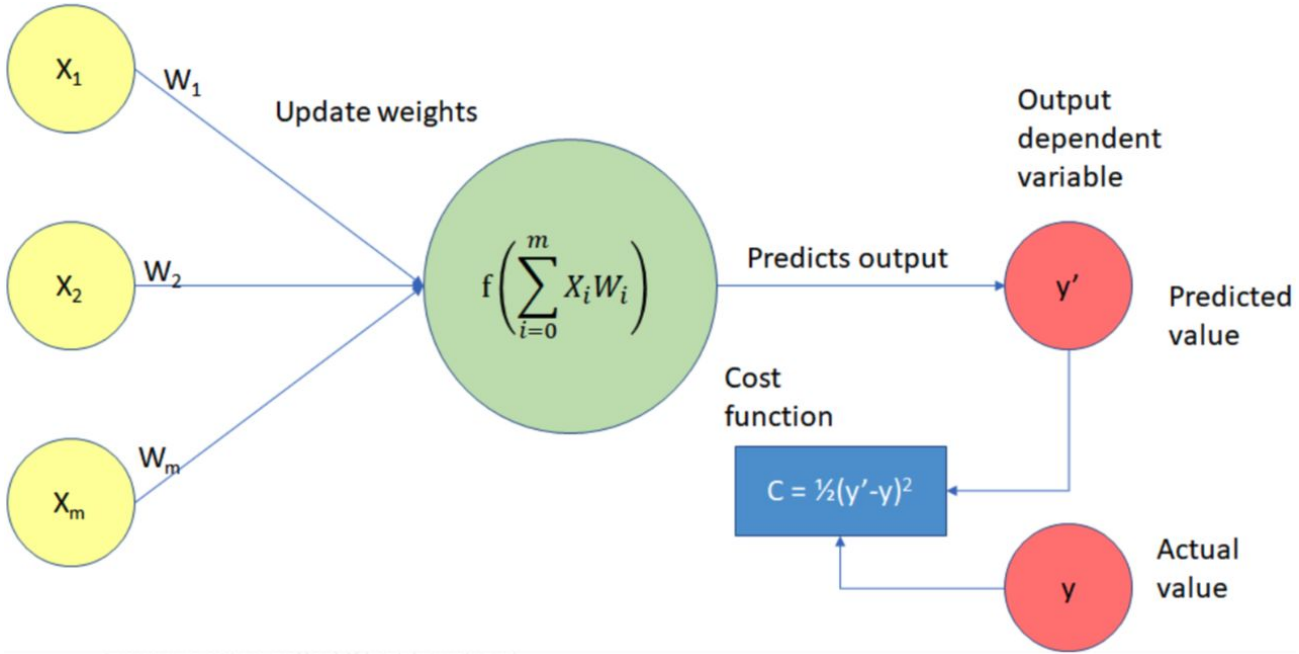
Chiwawa, Chiwawa

Muffin, Chiwawa



Cost Function

- A **cost function** is a measure of error between predictions and **ground truth**



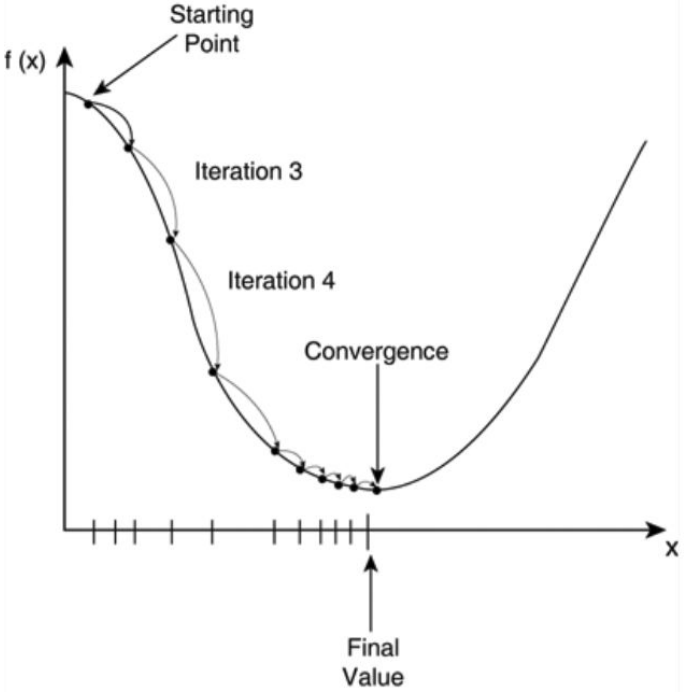
Cost Function

- A **cost function** is a measure of error between predictions and true values
- Guides the training process to find a set of weights that minimizes its value

Some examples:

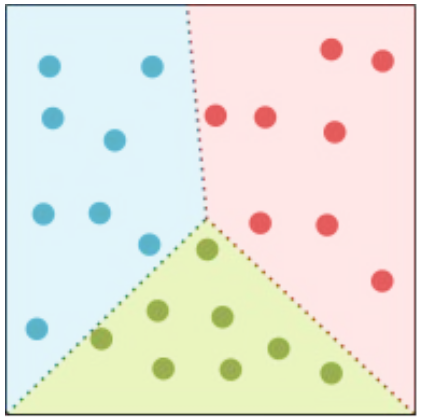
$$\text{MAE} = \underbrace{\frac{1}{n}}_{\text{test set}} \sum_{i=1}^{n} \underbrace{|y_i}_{\text{predicted vaue}} - \underbrace{\hat{y}_i|}_{\text{actual value}}$$

$$\text{MSE} = \underbrace{\frac{1}{n}}_{\text{test set}} \sum_{i=1}^{n} (\underbrace{y_i}_{\text{predicted vaue}} - \underbrace{\hat{y}_i}_{\text{actual value}})^2$$



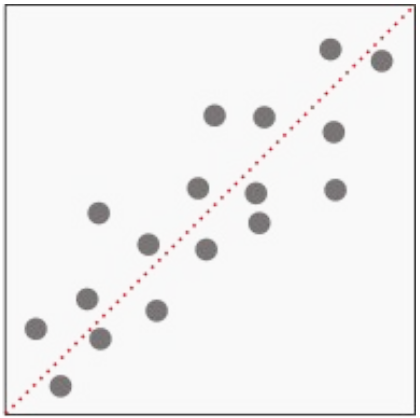
Cost Function

Two mains problems :



Classification

Cross Entropy
Hinge Loss



Regression

Mean Square Error
Mean Absolute Error

The choice of the **loss** is crucial !

What's important in the loss design?

- Adaptability to the problem
- Continuous and differentiable
- Numerically stable

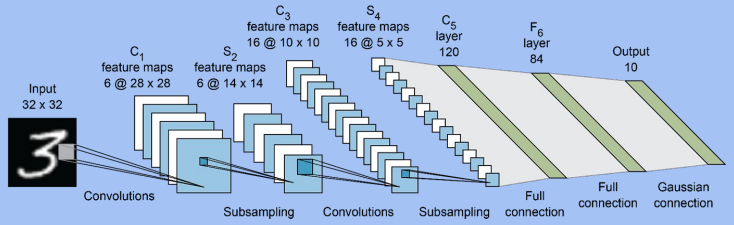
Deep Learning puzzle

Task

0

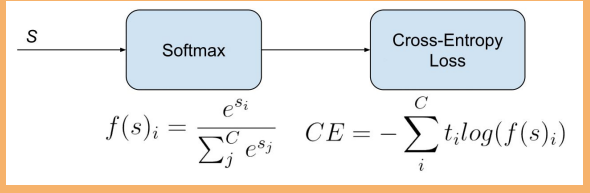
Digit Classification

Architecture



Data

Loss function

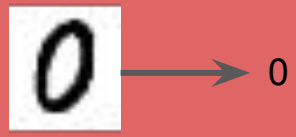


$$f(s)_i = \frac{e^{s_i}}{\sum_j^C e^{s_j}} \qquad CE = -\sum_i^C t_i log(f(s)_i)$$

Deep Learning puzzle

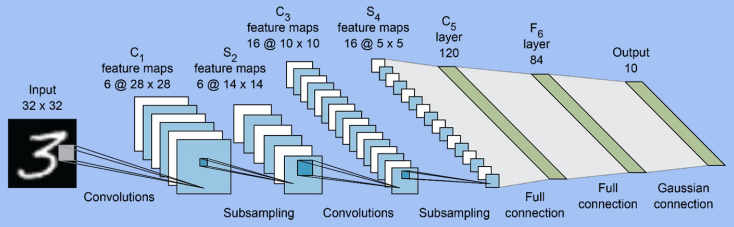
Deep Learning puzzle

Task



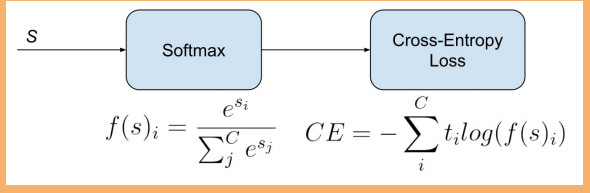
Digit Classification

Architecture



Optimization

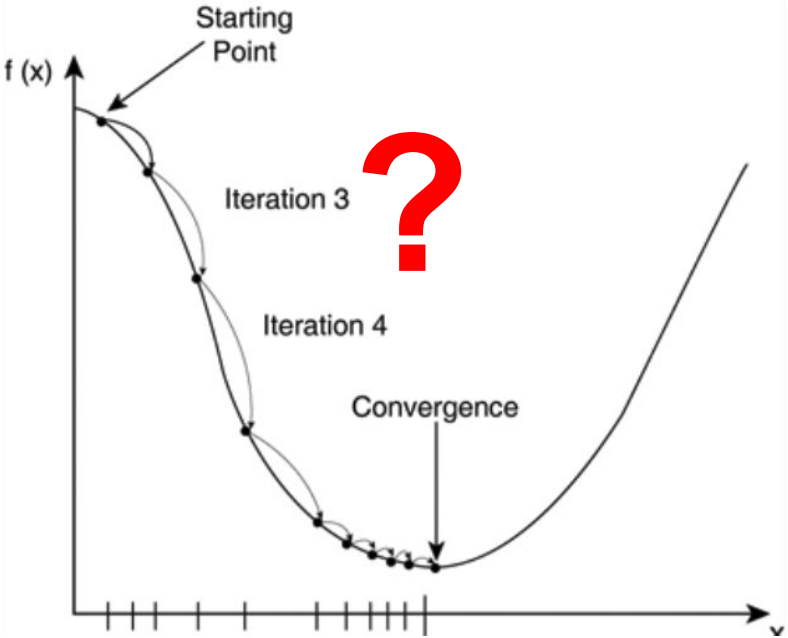
Data



Loss function

Optimization - Gradient Descent

- We have a function indicating how much error the model is doing.
- We want to reach the minimum of this function: Where the model is performing best
- How to optimize the weights to reach this point from our starting point?



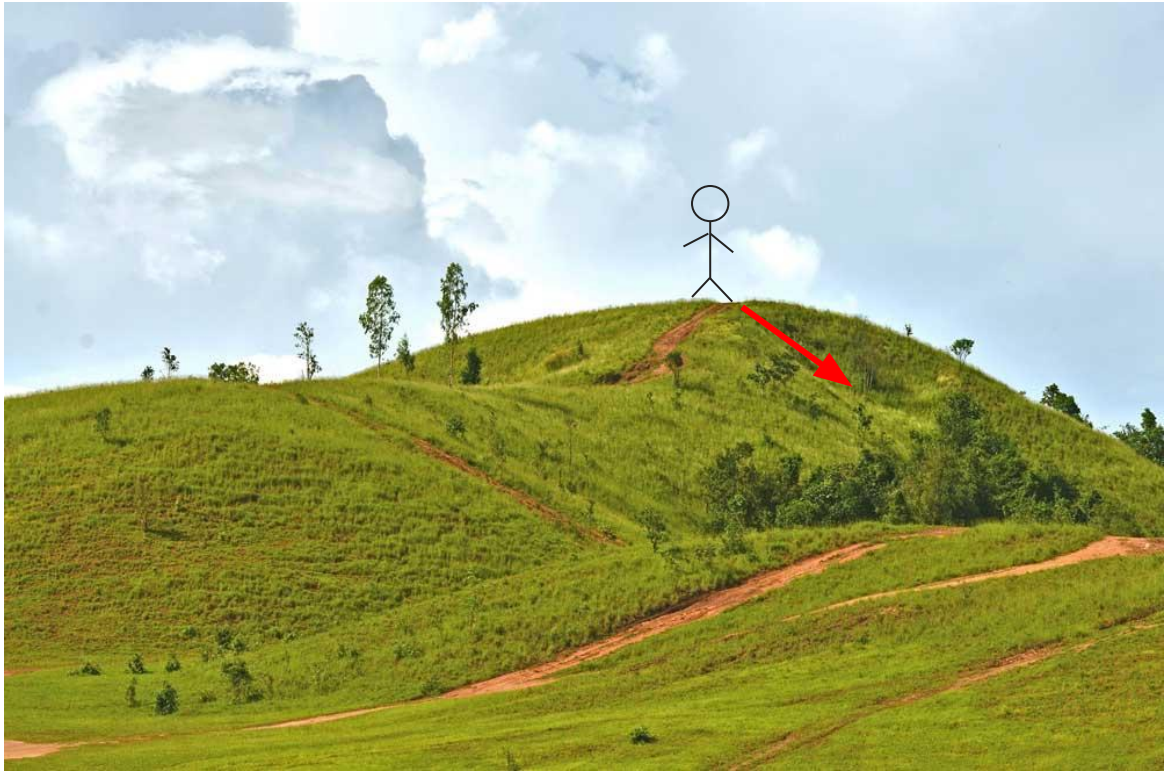
Optimization - Gradient Descent

You are at the top of the hill and want to go to the bottom

Optimization - Gradient Descent

You are at the top of the hill and want to go to the bottom

1. You look around and figure out the direction that slopes downward the most.



Optimization - Gradient Descent

You are at the top of the hill and want to go to the bottom

1. You look around and figure out the direction that slopes downward the most.
2. You take a step in that direction.

Optimization - Gradient Descent

You are at the top of the hill and want to go to the bottom

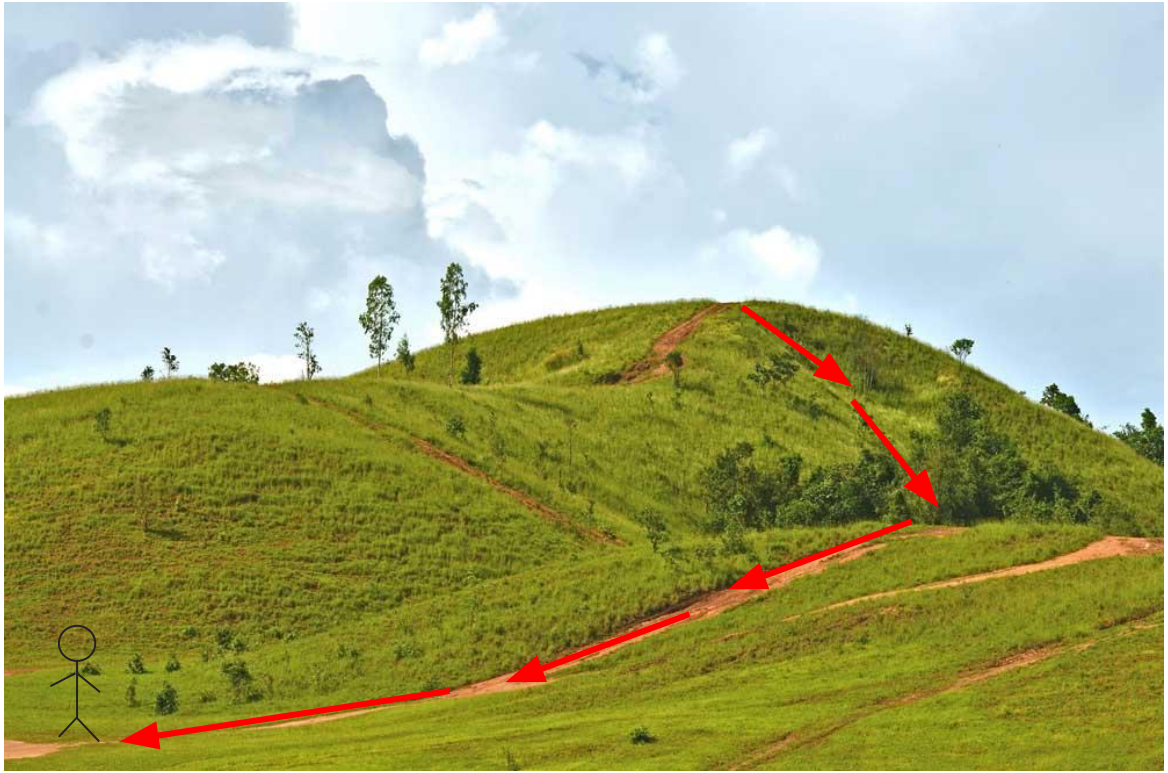
1. You look around and figure out the direction that slopes downward the most.
2. You take a step in that direction.
3. After taking the step, you check again where the ground is sloping downward the most and take another step in that direction.

Optimization - Gradient Descent

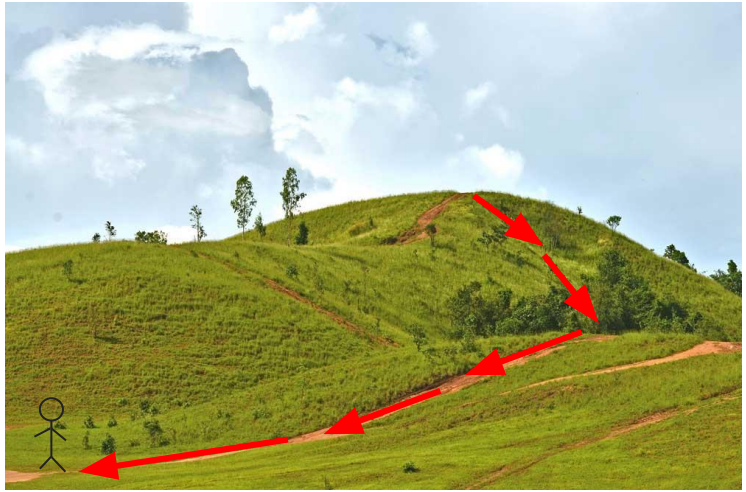
You are at the top of the hill and want to go to the bottom

1. You look around and figure out the direction that slopes downward the most.
2. You take a step in that direction.
3. After taking the step, you check again where the ground is sloping downward the most and take another step in that direction.

You keep repeating this until you get to the bottom of the valley.



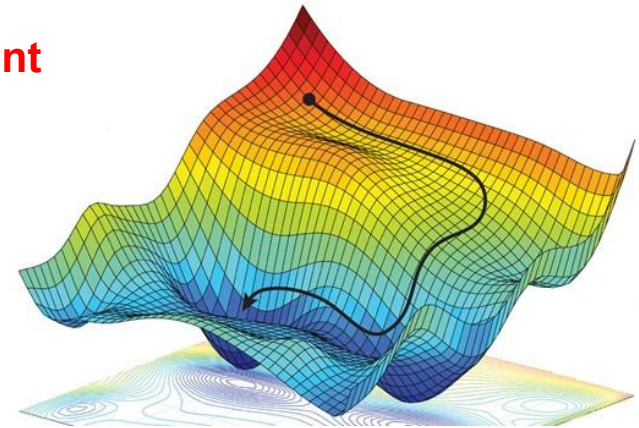
Optimization - Gradient Descent



The hill = Cost Function

Lowest Point = best solution minimizing error

Process of looking for the direction to move = **gradient descent**



Recall Gradient (Optimization)

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

The **derivative** shows the sensitivity of change of a function's output with respect to the input.
It can be seen as the slope of the function at a point.

In our case :

- The function is the error computed at the end of the Neural Network.
- We want to see how the error is impact by each parameter of the model →higher dimension than 1

Solution : use the **gradient**

Gradient of a function f at point a with $f : \mathbb{R}^3 \longrightarrow \mathbb{R}$

$$\nabla f(a) = \begin{bmatrix} \frac{\partial f}{\partial x}(a) \\ \frac{\partial f}{\partial y}(a) \\ \frac{\partial f}{\partial z}(a) \end{bmatrix}$$

Recall Gradient (Optimization)

What does this mean ? $\frac{\delta f}{\delta x}(a)$

Same meaning as the derivative of a function but for only one dimension when the function is in a multi-dimensional space

$\frac{\delta f}{\delta x}(a)$ $\frac{\text{How much f move with respect to the change in x}}{\text{A very small change in x (ex : 0.0001)}}$

The slope of the function f with respect to the input x at point a

Recall Gradient (Optimization)

Example :

$$f(x,y,z) = x^2 + y * z \longrightarrow \nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \\ \frac{\partial f}{\partial z} \end{bmatrix} = \begin{bmatrix} 2x \\ z \\ y \end{bmatrix}$$

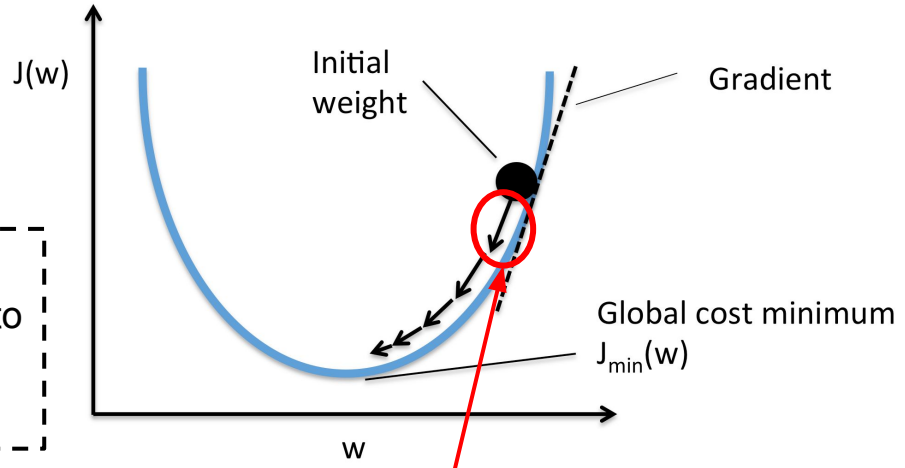
- We obtain the slope of the function f in each direction (x, y, z)
- The direction will be replaced by each parameter of the model (w and b)

Optimization - Gradient Descent

- Find the minimum of the cost function *C*
- The Negative gradient : $-\nabla C$ points in the direction where the function decreases most rapidly
- Calculate new weights: $W^+ = W - \eta \nabla C$

$$\begin{bmatrix} w_1^+ \\ w_2^+ \\ \vdots \\ w_n^+ \end{bmatrix} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix} - \eta \begin{bmatrix} \frac{\partial C}{\partial w_1} \\ \frac{\partial C}{\partial w_2} \\ \vdots \\ \frac{\partial C}{\partial w_n} \end{bmatrix}$$

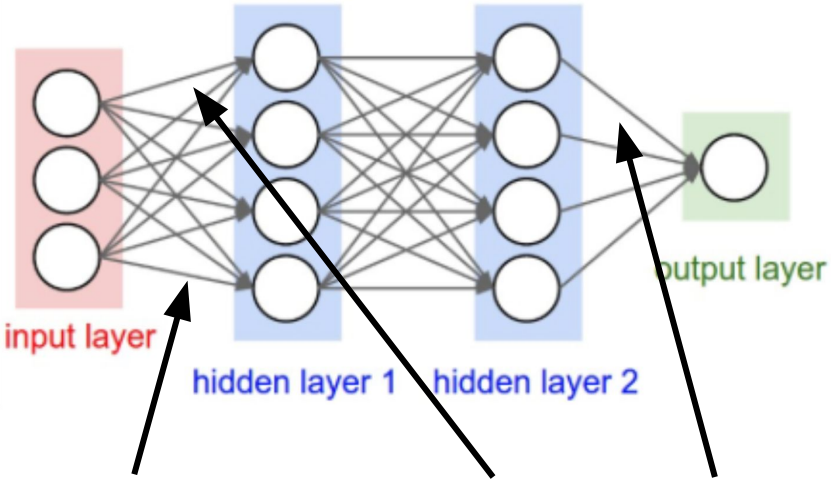
How much the cost function is sensitive to a change for the weight w_1



- η is called the **learning rate** : increase or decrease the length of the steps

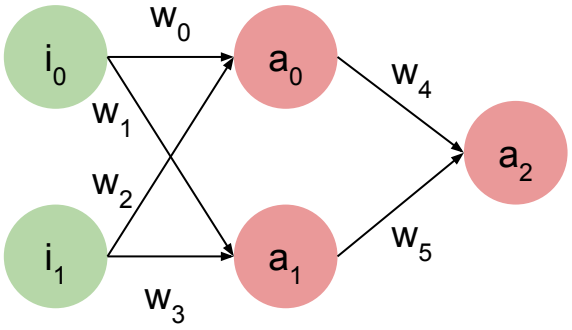
Optimization - Backpropagation

- We have a function indicating how much error the model is doing.
- We now how to reach the minimum
- BUT by how much each part of the network contributed to the error and should be adjusted?



Optimization - Backpropagation

Step by step going backward:



$$z_2 = w_4 . a_0 + w_5 . a_1 + b_2$$

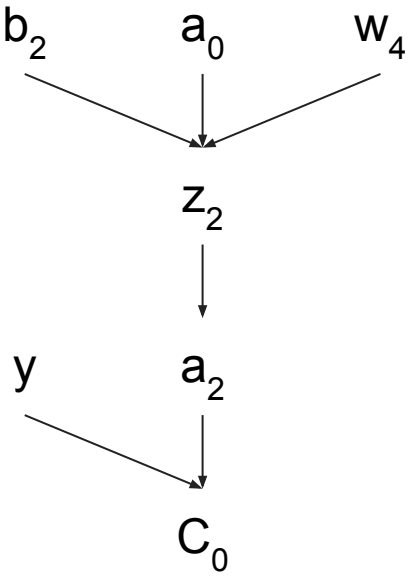
$$a_2 = \sigma(z_2)$$ output

$$C_0 = (a_2 - y)^2$$ Cost Function

- Last Layer: adjust a_2 to be as close as possible to y

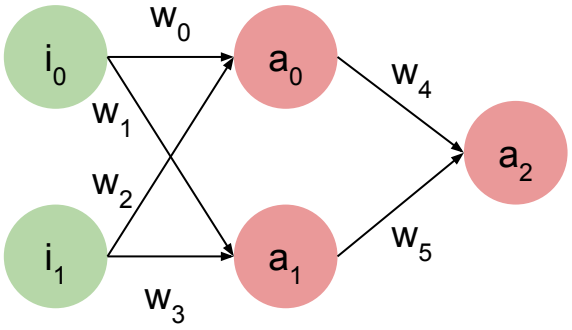
The layer can directly observe the error it did

Can directly know by how much these weights affected the error: computing the gradient of the error on w_4, w_5, b_2



Optimization - Backpropagation

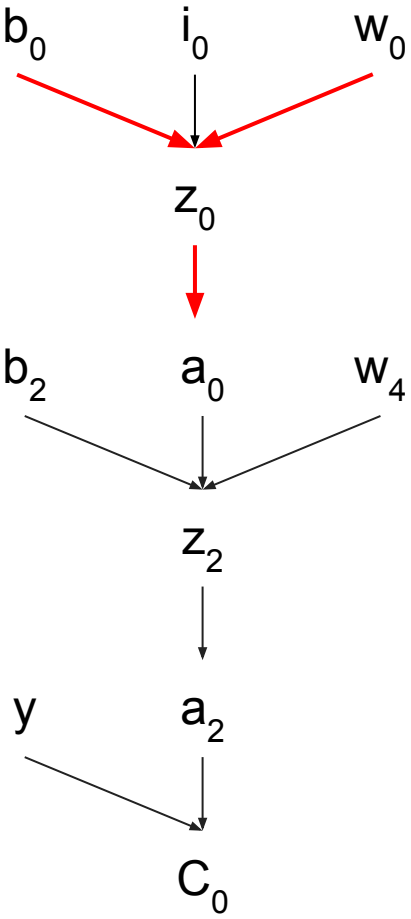
Step by step going backward:



$$z_2 = w_4 . a_0 + w_5 . a_1 + b_2$$

$$a_2 = \sigma(z_2)$$ output

$$C_0 = (a_2 - y)^2$$ Cost Function



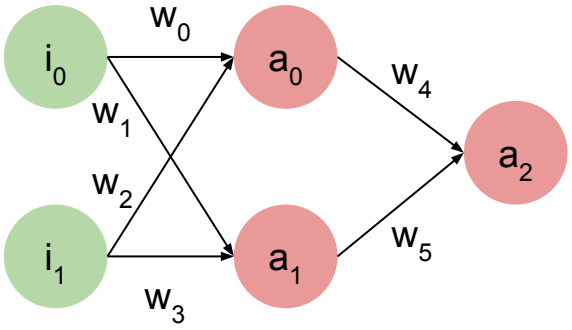
- First Layer: adjust a_0 and a_1 so that a_0 get closer to y

Cannot observe directly the error because one layer after him

Propagate the error backward so that it can adjust his weights, BUT will see the error with respect to values from last layer

propagate C_0 back to w_0 through a_2, z_2, a_0, z_0: **chain rule**

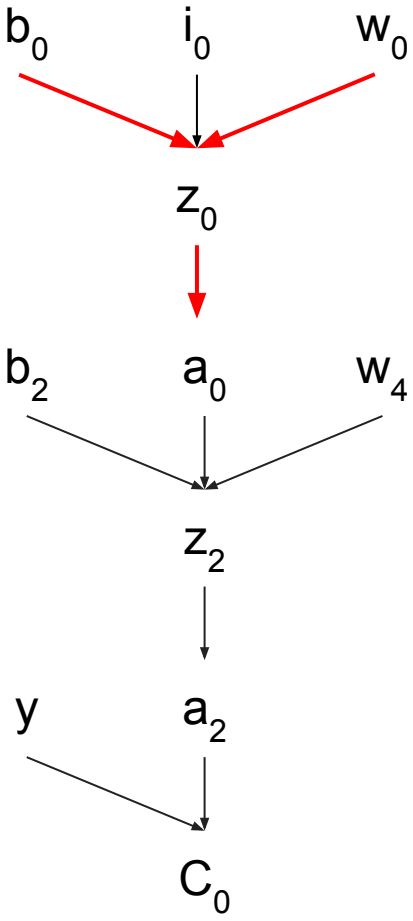
Optimization - Backpropagation



$$z_2 = w_4 . a_0 + w_5 . a_1 + b_2$$

$$a_2 = \sigma(z_2)$$

$$C_0 = (a_2 - y)^2$$



Adjust w_0:

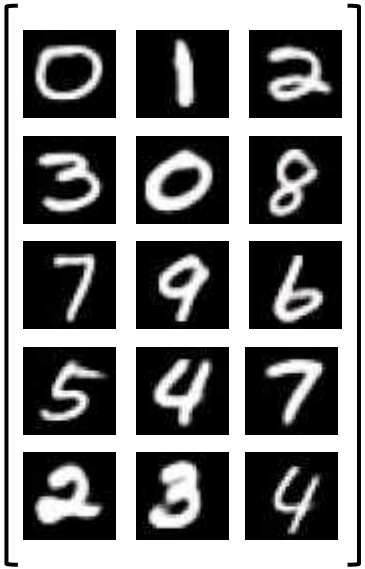
$$\frac{\delta C_0}{\delta w_0} = \frac{\delta z_0}{\delta w_0} \frac{\delta a_0}{\delta z_0} \frac{\delta z_2}{\delta a_0} \frac{\delta a_2}{\delta z_2} \frac{\delta C_0}{\delta a_2}$$

How much w_0 is responsible of the error?

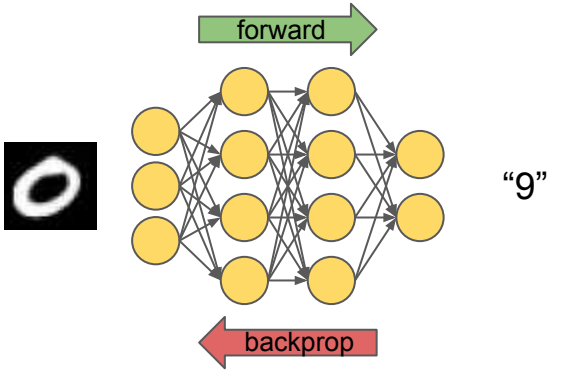
Optimization - Backpropagation

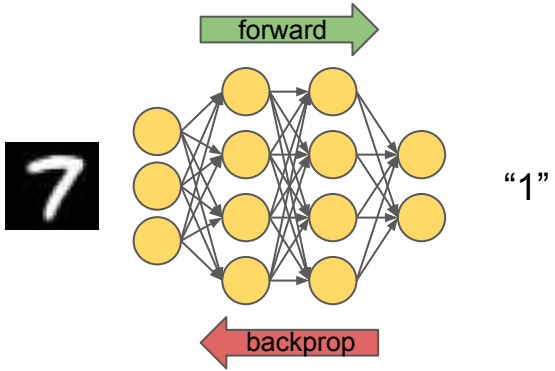
Compute Forward + Backprop for each data point : **Stochastic Gradient Descent**

- Bad idea

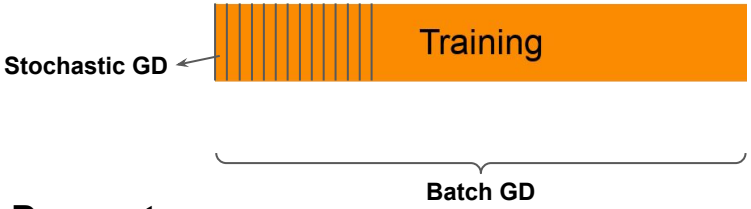


Dataset



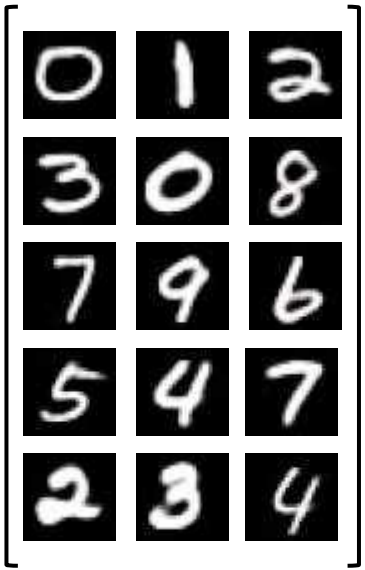


Optimization - Backpropagation

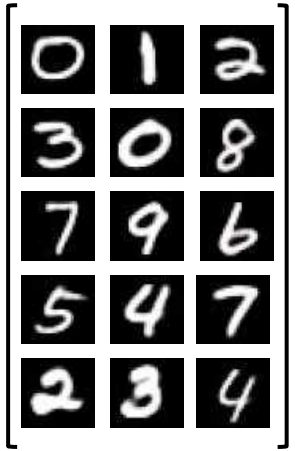


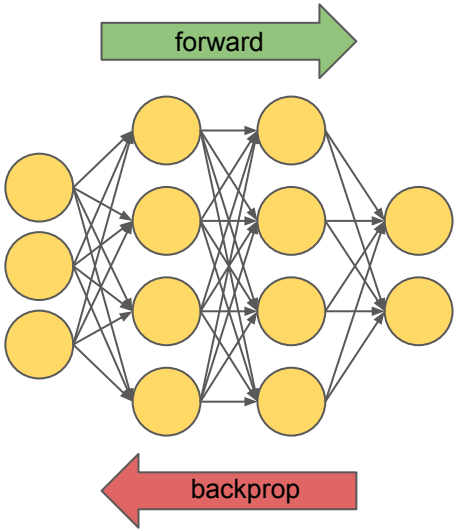
Compute Forward + Backprop for all dataset once : **Batch Gradient Descent**

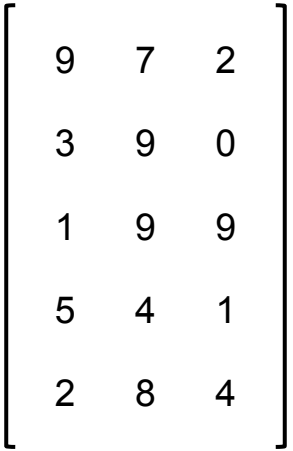
- Not Feasible



Dataset



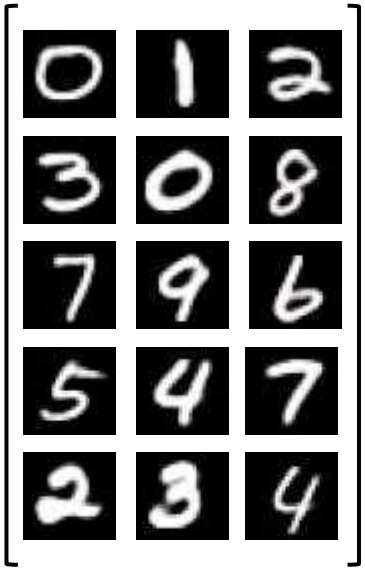




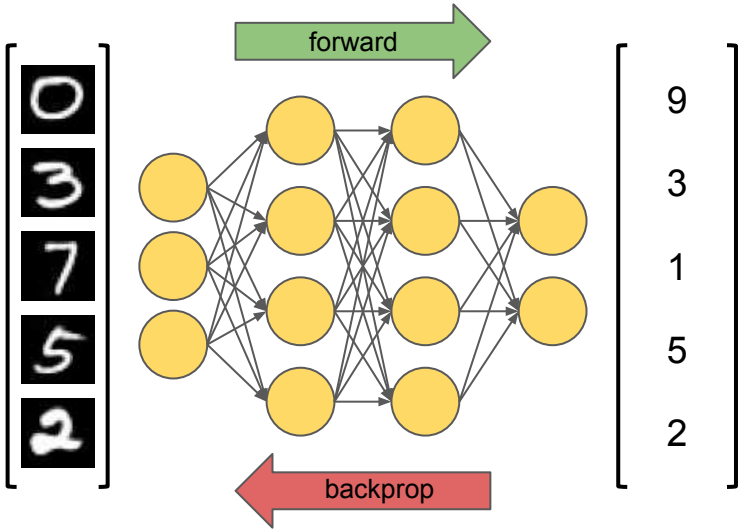
Optimization - Backpropagation

Compute Forward + Backprop for a part (**Batch**) of the dataset : **Mini-Batch Gradient Descent**

- Good idea



Dataset



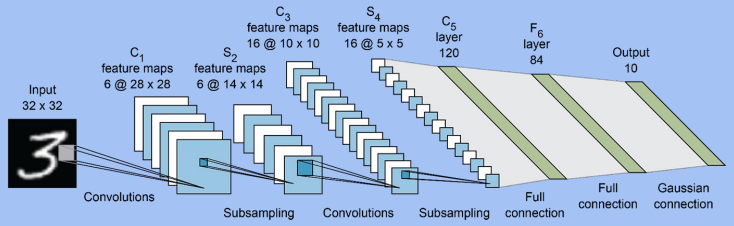
How to learn ?

Task

0

Digit Classification

Architecture

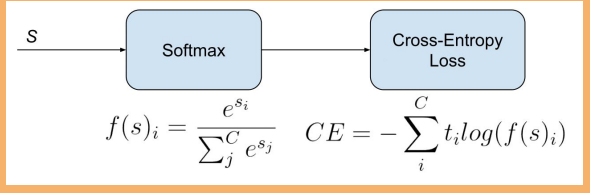


Optimization

∇

backpropagation

Data

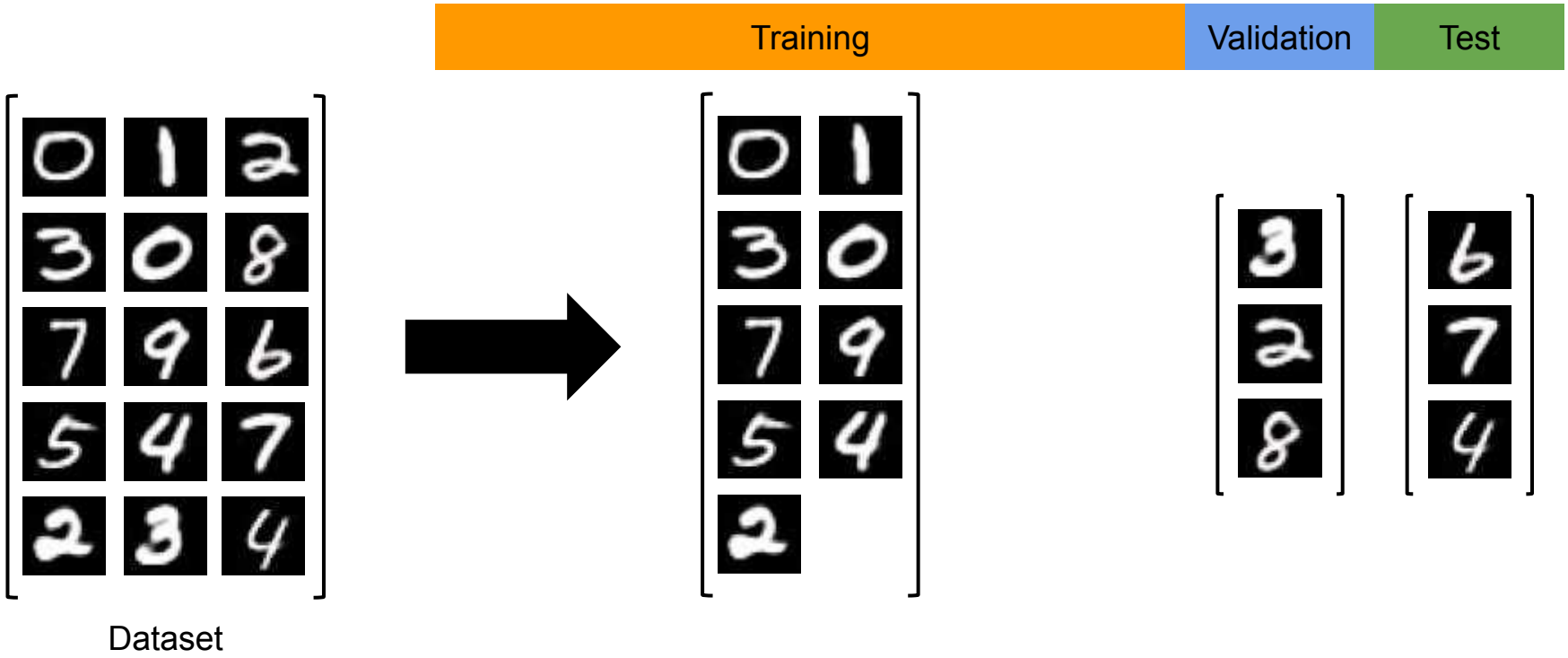


$$f(s)_i = \frac{e^{s_i}}{\sum_j^C e^{s_j}} \quad CE = -\sum_i^C t_i log(f(s)_i)$$

Loss function

Deep Learning in practice

Divide your dataset in 3 parts

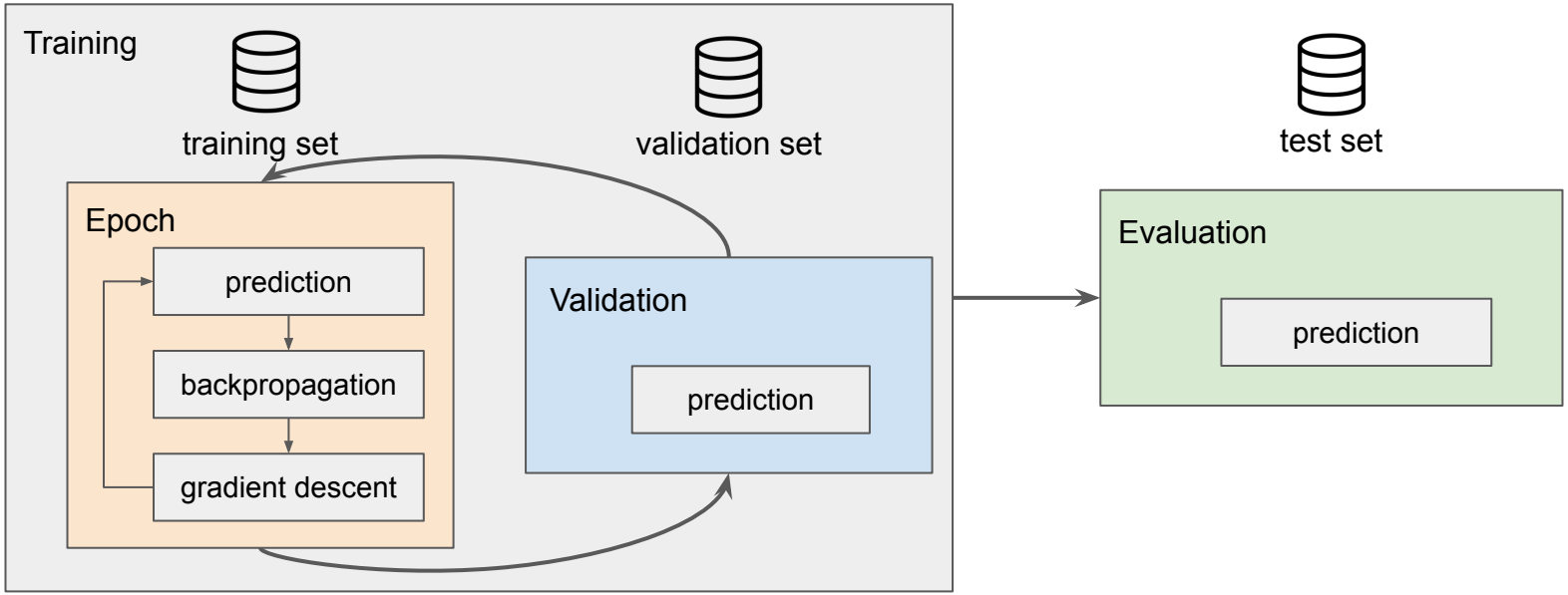


Deep Learning in practice

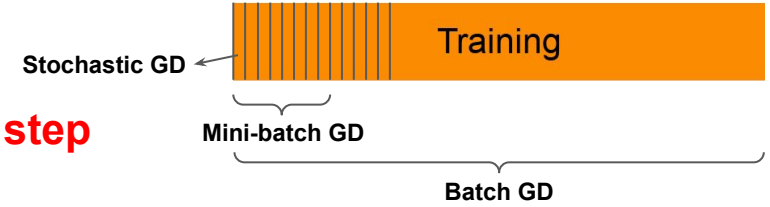
Training: optimization of the model

Validation: testing generalization

Evaluation: test generalization on another dataset at the end of training

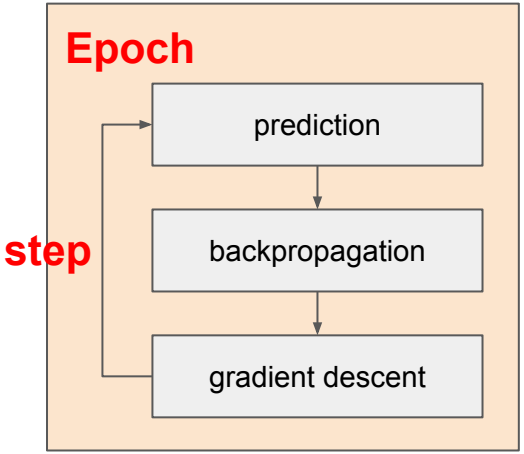


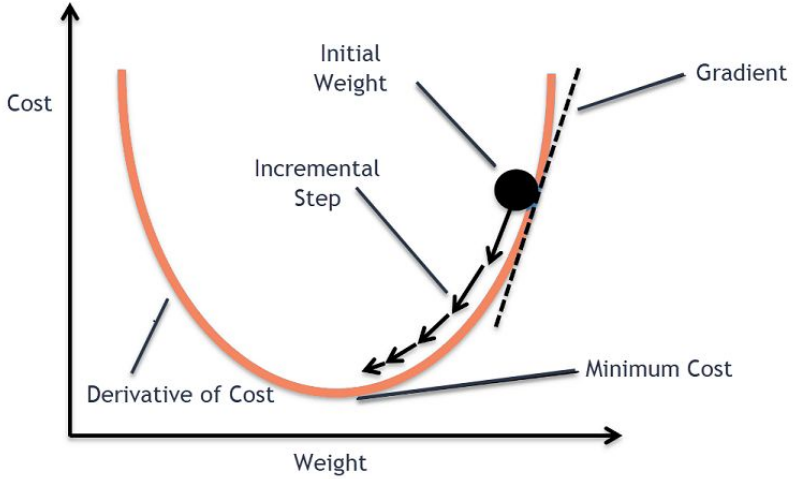
Deep Learning in practice



SGD is compute on the **training set** at the end of each **step**

Epoch: one pass through the dataset

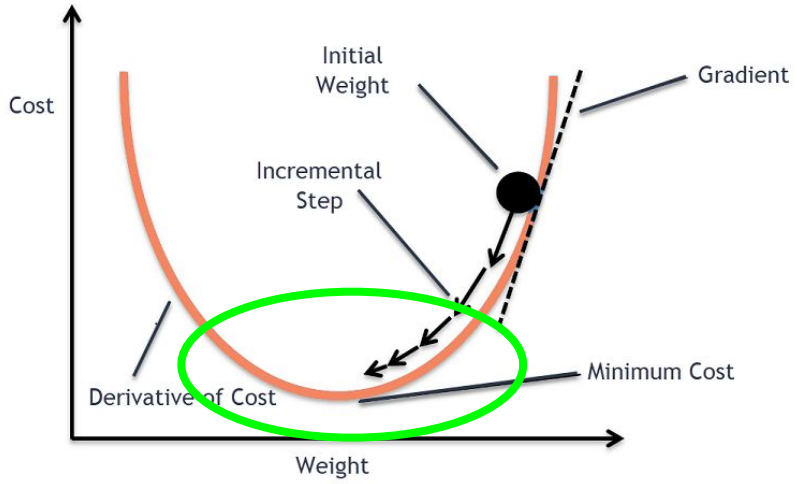




Deep Learning in practice

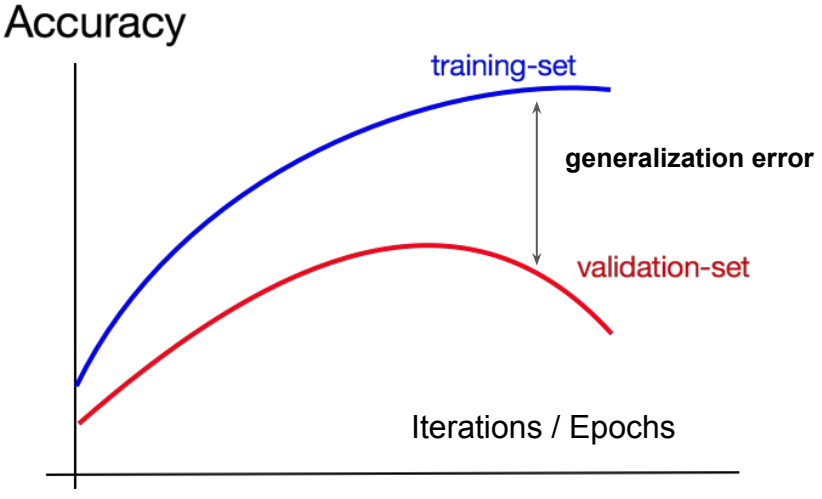
How do we know when to stop learning ?

When cost function reaches the minimum :
the model has **converged**



Model can **overfit** on training set

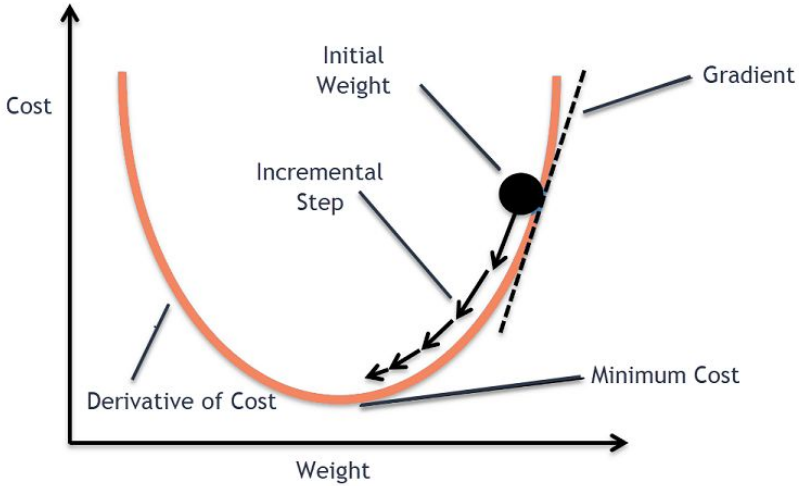
Stop the training when error on validation set reaches the minimum

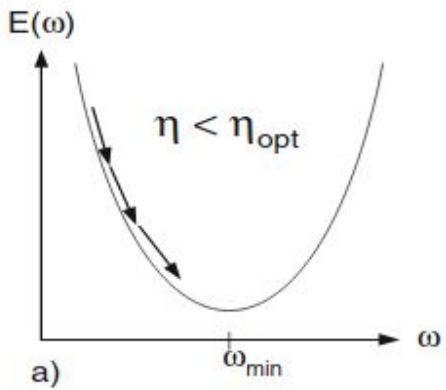


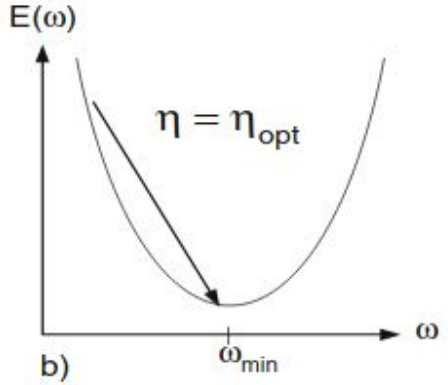
Deep Learning in practice

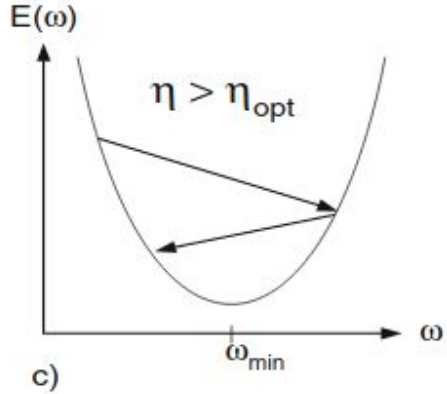
How to find the **learning rate** ? η

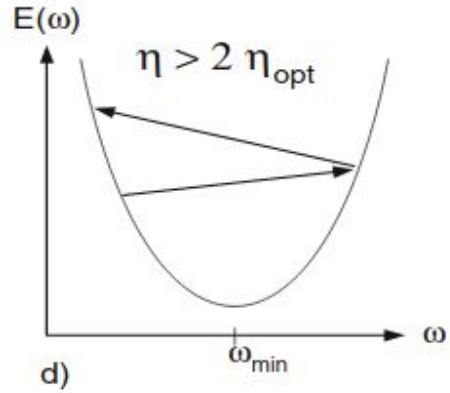
$$\theta \leftarrow \theta - \eta \frac{\partial \mathcal{L}}{\partial \theta}$$





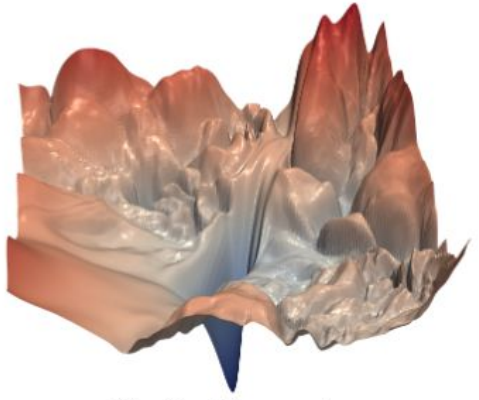






Deep Learning in practice

$$\theta \leftarrow \theta - \eta \frac{\partial \mathcal{L}}{\partial \theta}$$



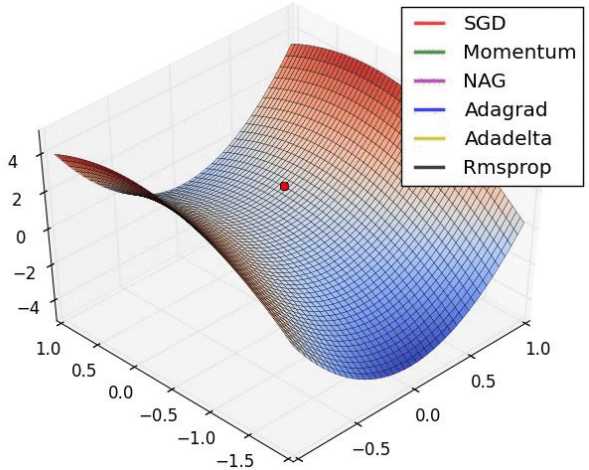
Loss landscapes are not easy to navigate for optimizers

Ideas:

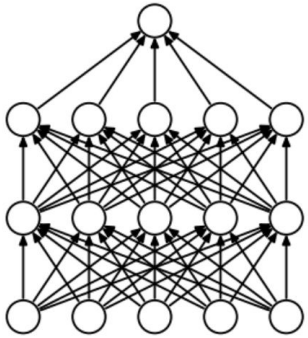
- Use the gradient history of previous timesteps to inform GD in future timesteps
- Adapt the learning rate to each parameter

This is a 2 parameter example!
Imagine millions

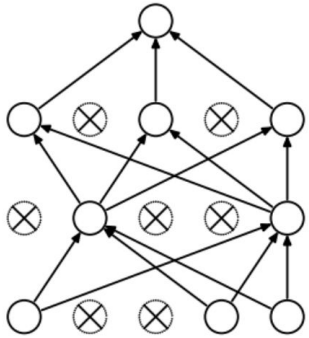
Adam optimizer is an extension of SGD which makes use of these two ideas. It is currently the most used optimizer in DL after SGD.



Deep Learning in practice



(a) Standard Neural Net



(b) After applying dropout.

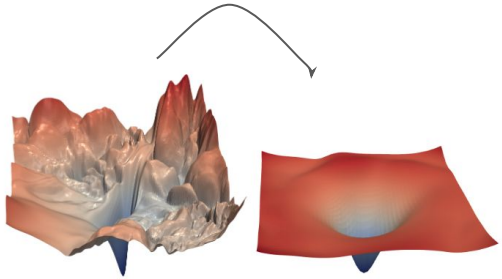
Additional constraints to reduce overfitting

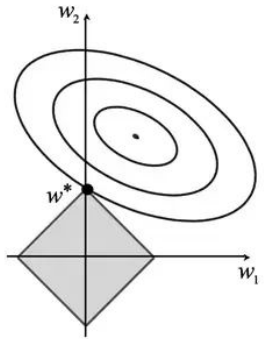
Dropout: stochastically dropping weights during inference

Early stopping: stopping the training as soon as the validation loss starts increasing

Weight penalties (weight decay): L1 norm (Lasso) / L2 norm (ridge). Terms added to the loss.

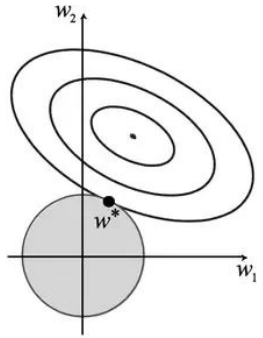
Data augmentations: artificially boosting the number of training samples





L1

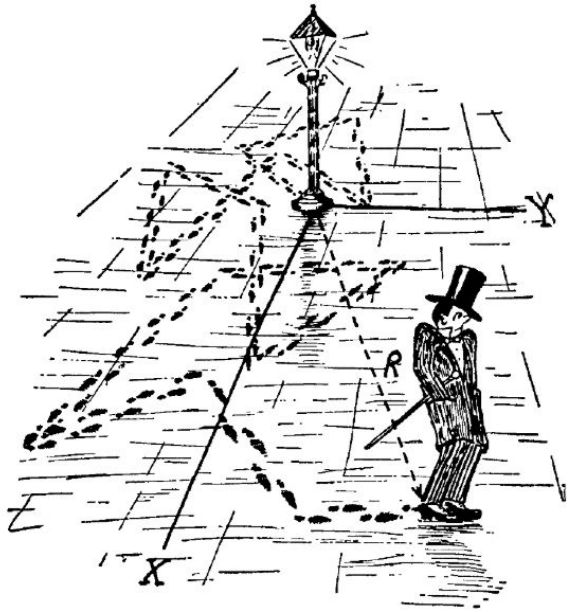
Sparse weights



L2

Smaller weight values

Deep Learning in practice

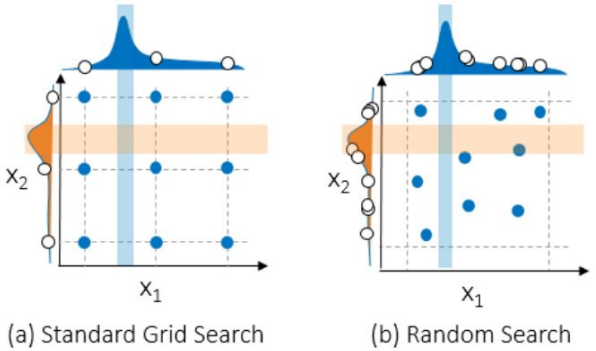


All parameters and settings that are set before training:

- Architectural choices: types and number of layers, size of each layer
- Losses/regularization: weights, additional constants
- Optimization: batch size, learning rate, iterations/epochs, schedule

Hyperparameter search. Another optimization problem ?

- Often done manually.
- When resources are available, large scale search is possible

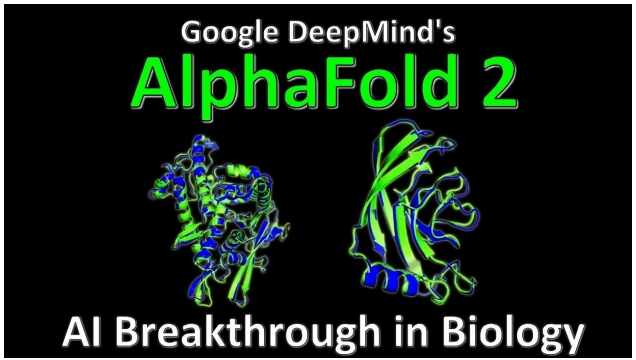


(a) Standard Grid Search (b) Random Search

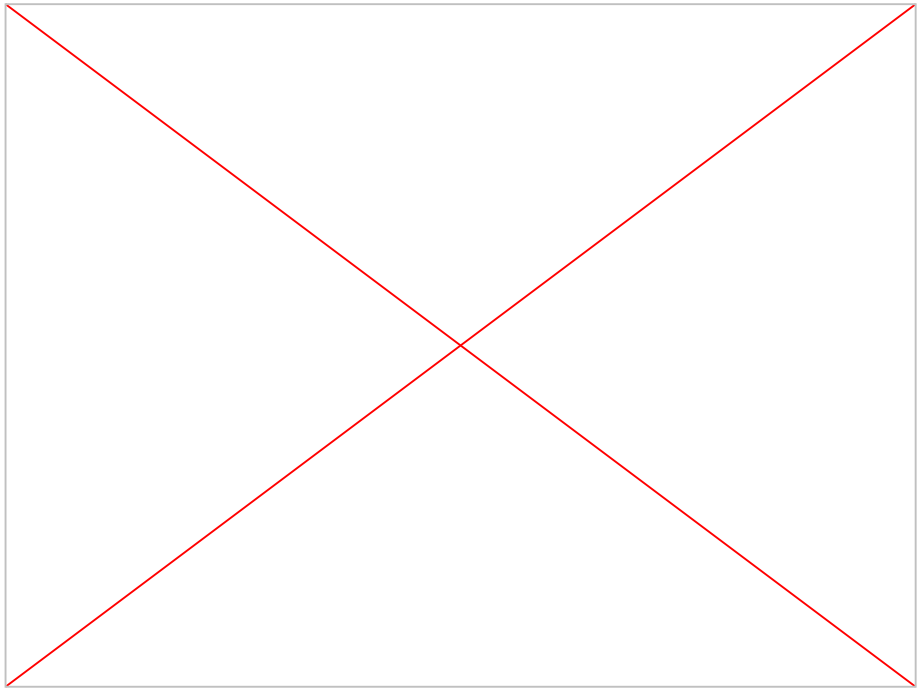
Practical examples

GPT-4

DALL·E 2



Midjourney



Palm-E

Additional resources

3Blue1Brown Neural Networks videos:

https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi

Blog post:

http://neuralnetworksanddeeplearning.com/chap1.html

https://towardsdatascience.com/part-2-gradient-descent-and-backpropagation-bf90932c066a

https://towardsdatascience.com/a-concise-history-of-neural-networks-2070655d3fec#.ekc89166m

https://people.idsia.ch//~juergen/who-invented-backpropagation.html

Practical Work